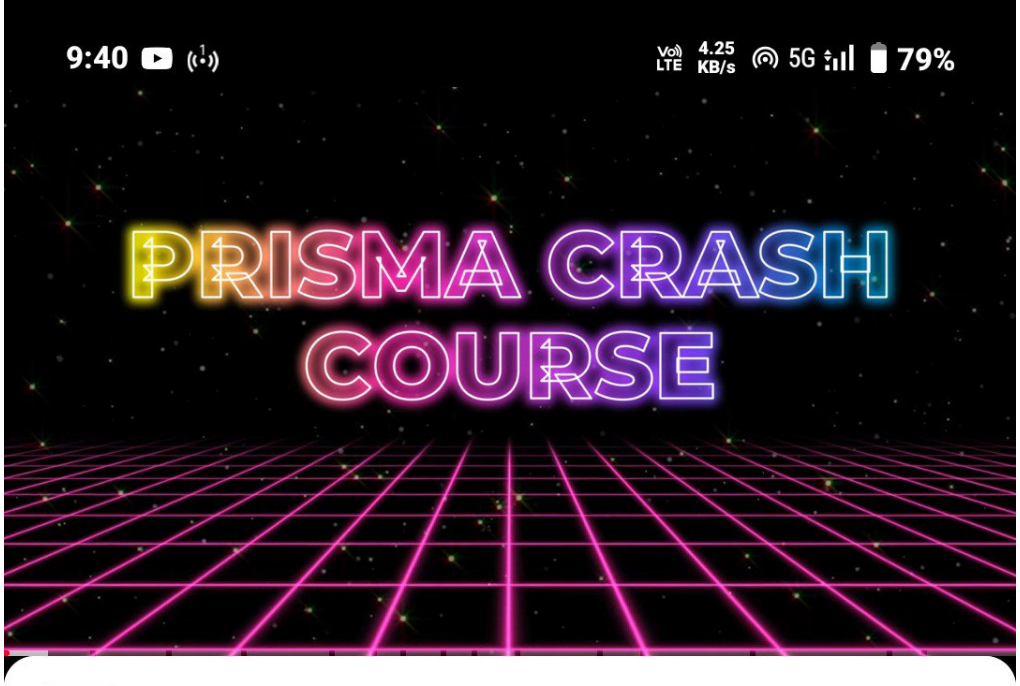


Prisma ORM in One Video 🔥 Setup, Migrations, CRUD & Best Practices

31K views 8mo ago ...more

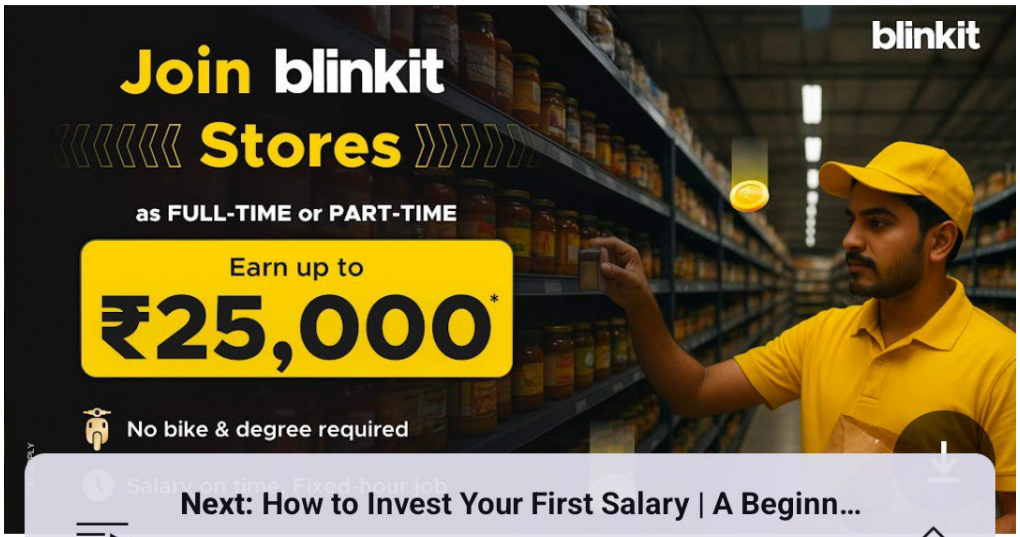
Thapa Technical 738K

Join

769 | Share | Remix | Download

Comments 56

A Sir we don't want crash course we want full course like other course please please



Next: How to Invest Your First Salary | A Beginn...

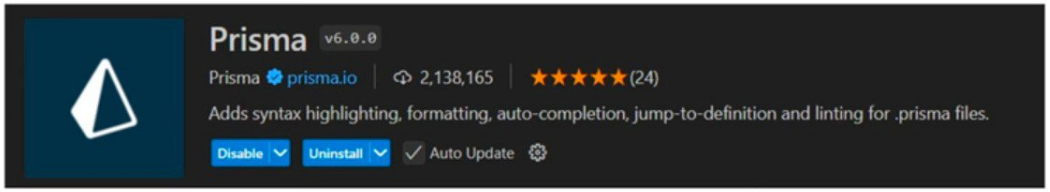
Very IMP • 29/37

Blinkit Picker Onboarding

आता लगेच जॉइन करा!

Introduction to Prisma ORM

- Prisma is an open-source ORM (Object-Relational Mapping) tool for Node.js and TypeScript that simplifies database interaction.
- It helps you interact with databases using a type-safe query builder and eliminates the need to write raw SQL queries.
- Prisma provides Prisma Client, a powerful auto-generated query builder, and Prisma Migrate for database schema migrations.
- https://www.prisma.io/docs/getting-started/setup-prisma
- Install **Prisma** Vscode Extension



Prisma Client

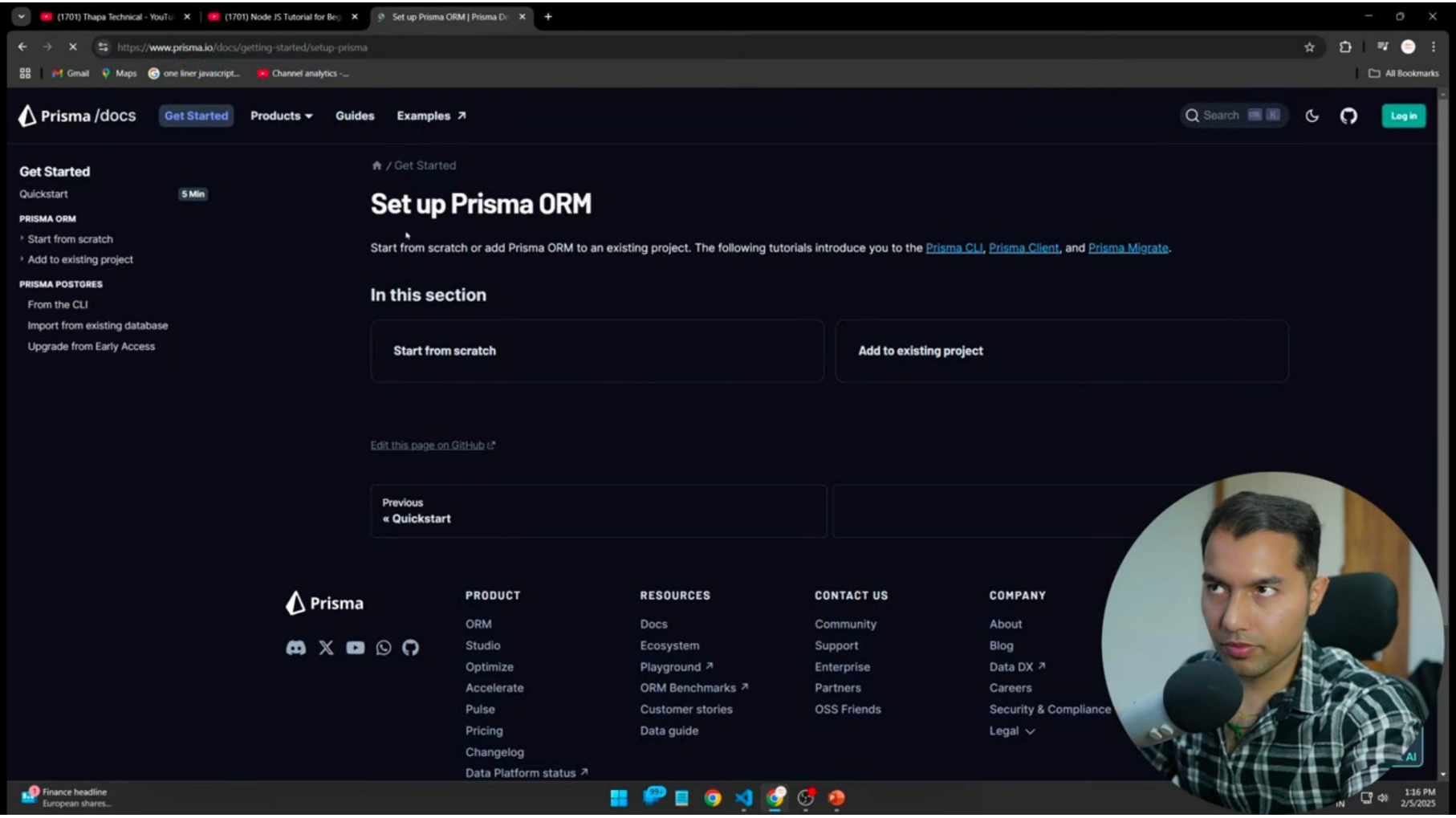
Auto-generated and type-safe database client.

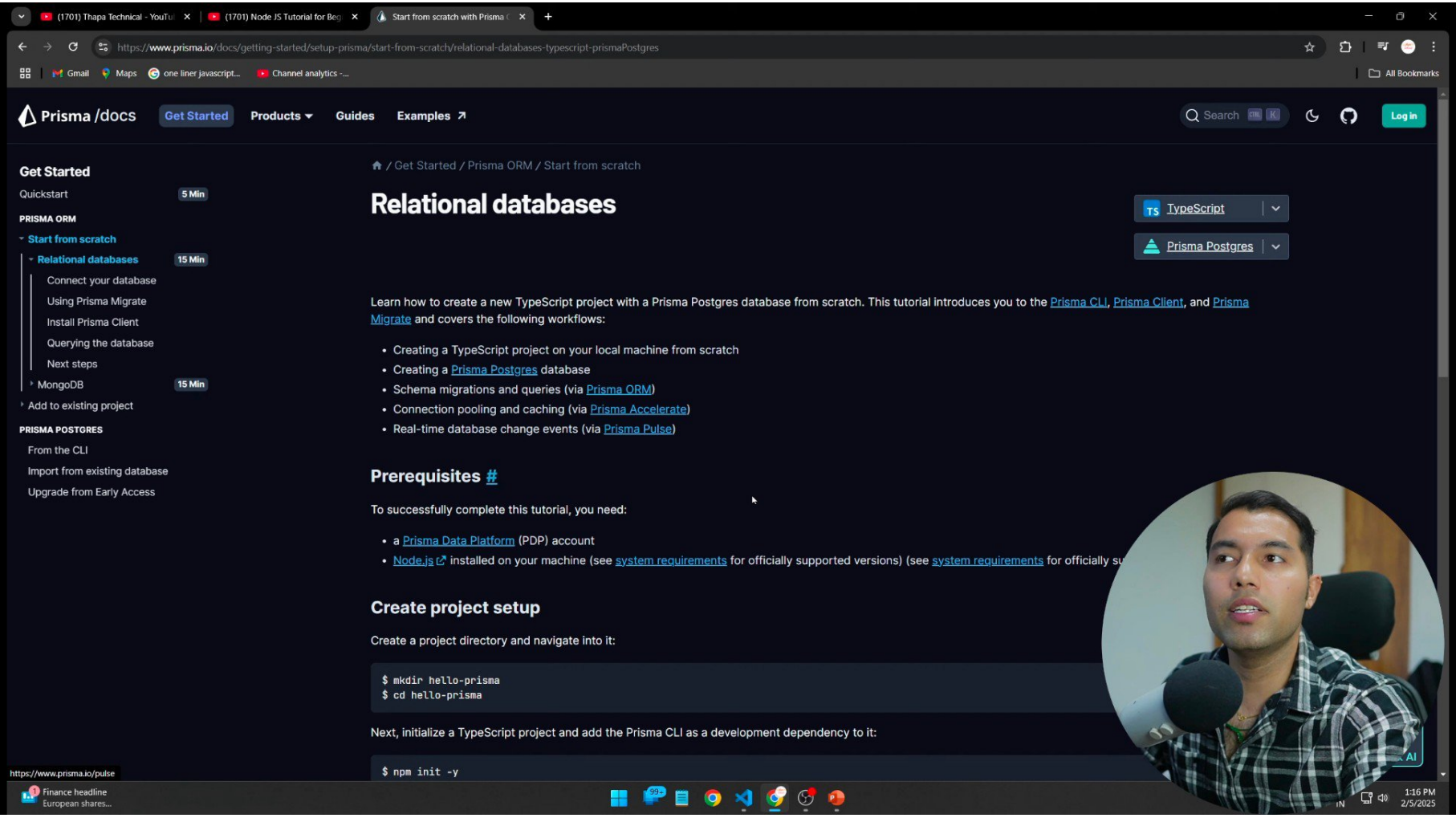
Prisma Migrate

Declarative data modeling and customizable migrations.

Prisma Studio

Modern UI to view and edit your application data.


(1701) Thapa Technical - YouTu
(1701) Node JS Tutorial for Beg
Set up Prisma ORM | Prisma Do
https://www.prisma.io/docs/getting-started/setup-prisma
Gmail
Maps
one liner javascript...
Channel analytics -...
All Bookmarks
Prisma /docs
Get Started
Products
Guides
Examples
Search
Log in
Get Started
Quickstart
5 Min
PRISMA ORM
Start from scratch
Add to existing project
PRISMA POSTGRES
From the CLI
Import from existing database
Upgrade from Early Access
/ Get Started
Set up Prisma ORM
Start from scratch or add Prisma ORM to an existing project. The following tutorials introduce you to the Prisma CLI, Prisma Client, and Prisma Migrate.
In this section
Start from scratch
Add to existing project
Edit this page on GitHub
Previous
« Quickstart
Prisma
PRODUCT
ORM
Studio
Optimize
Accelerate
Pulse
Pricing
Changelog
Data Platform status
RESOURCES
Docs
Ecosystem
Playground
ORM Benchmarks
Customer stories
Data guide
CONTACT US
Community
Support
Enterprise
Partners
OSS Friends
COMPANY
About
Blog
Data DX
Careers
Security & Compliance
Legal
AI
Finance headline
European shares...
1:16 PM
2/5/2025


(1701) Thapa Technical - YouTu
(1701) Node JS Tutorial for Beg
Start from scratch with Prisma
https://www.prisma.io/docs/getting-started/setup-prisma/start-from-scratch/relational-databases-typescript-prismaPostgres
Gmail
Maps
one liner javascript...
Channel analytics -...
All Bookmarks
Prisma /docs
Get Started
Products
Guides
Examples
Search
Log in
Get Started
Quickstart
5 Min
PRISMA ORM
Start from scratch
Relational databases
15 Min
Connect your database
Using Prisma Migrate
Install Prisma Client
Querying the database
Next steps
MongoDB
15 Min
Add to existing project
PRISMA POSTGRES
From the CLI
Import from existing database
Upgrade from Early Access
/ Get Started / Prisma ORM / Start from scratch
Relational databases
TypeScript
Prisma Postgres
Learn how to create a new TypeScript project with a Prisma Postgres database from scratch. This tutorial introduces you to the Prisma CLI, Prisma Client, and Prisma Migrate and covers the following workflows:
Creating a TypeScript project on your local machine from scratch
Creating a Prisma Postgres database
Schema migrations and queries (via Prisma ORM)
Connection pooling and caching (via Prisma Accelerate)
Real-time database change events (via Prisma Pulse)
Prerequisites #
To successfully complete this tutorial, you need:
a Prisma Data Platform (PDP) account
Node.js installed on your machine (see system requirements for officially supported versions) (see system requirements for officially su
Create project setup
Create a project directory and navigate into it:
$ mkdir hello-prisma
$ cd hello-prisma
Next, initialize a TypeScript project and add the Prisma CLI as a development dependency to it:
$ npm init -y
https://www.prisma.io/pulse
Finance headline
European shares...
1:16 PM
2/5/2025

(1701) Thapa Technical - YouTu | (1701) Node JS Tutorial for Beg | Start from scratch with Prisma

https://www.prisma.io/docs/getting-started/setup-prisma/start-from-scratch/relational-databases-typescript-prismaPostgres

Gmail | Maps | one liner javascript... | Channel analytics -... | All Bookmarks

Prisma /docs | Get Started | Products | Guides | Examples | Search Ctrl K | Log in

Get Started

- Quickstart — 5 Min

PRISMA ORM

- Start from scratch
  - Relational databases — 15 Min
    - Connect your database
    - Using Prisma Migrate
    - Install Prisma Client
    - Querying the database
    - Next steps
  - MongoDB — 15 Min
- Add to existing project

PRISMA POSTGRES

- From the CLI
- Import from existing database
- Upgrade from Early Access

Create project setup

Create a project directory and navigate into it:

```
$ mkdir hello-prisma
$ cd hello-prisma
```

Next, initialize a TypeScript project and add the Prisma CLI as a development dependency to it:

```
$ npm init -y
$ npm install prisma typescript tsx @types/node --save-dev
```

This creates a `package.json` with an initial setup for your TypeScript app.

Next, initialize TypeScript:

```
$ npx tsc --init
```

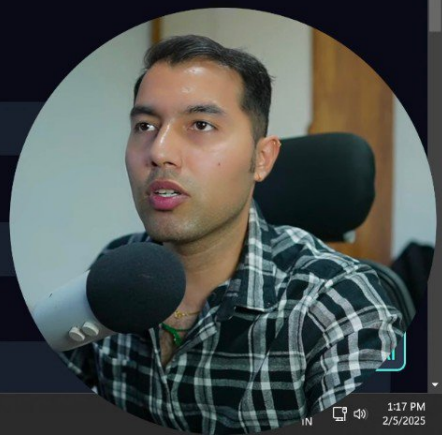
You can now invoke the Prisma CLI by prefixing it with `npx`:

```
$ npx prisma
```

Next, set up your Prisma ORM project by creati ollowing command:

```
$ npx prisma init
```

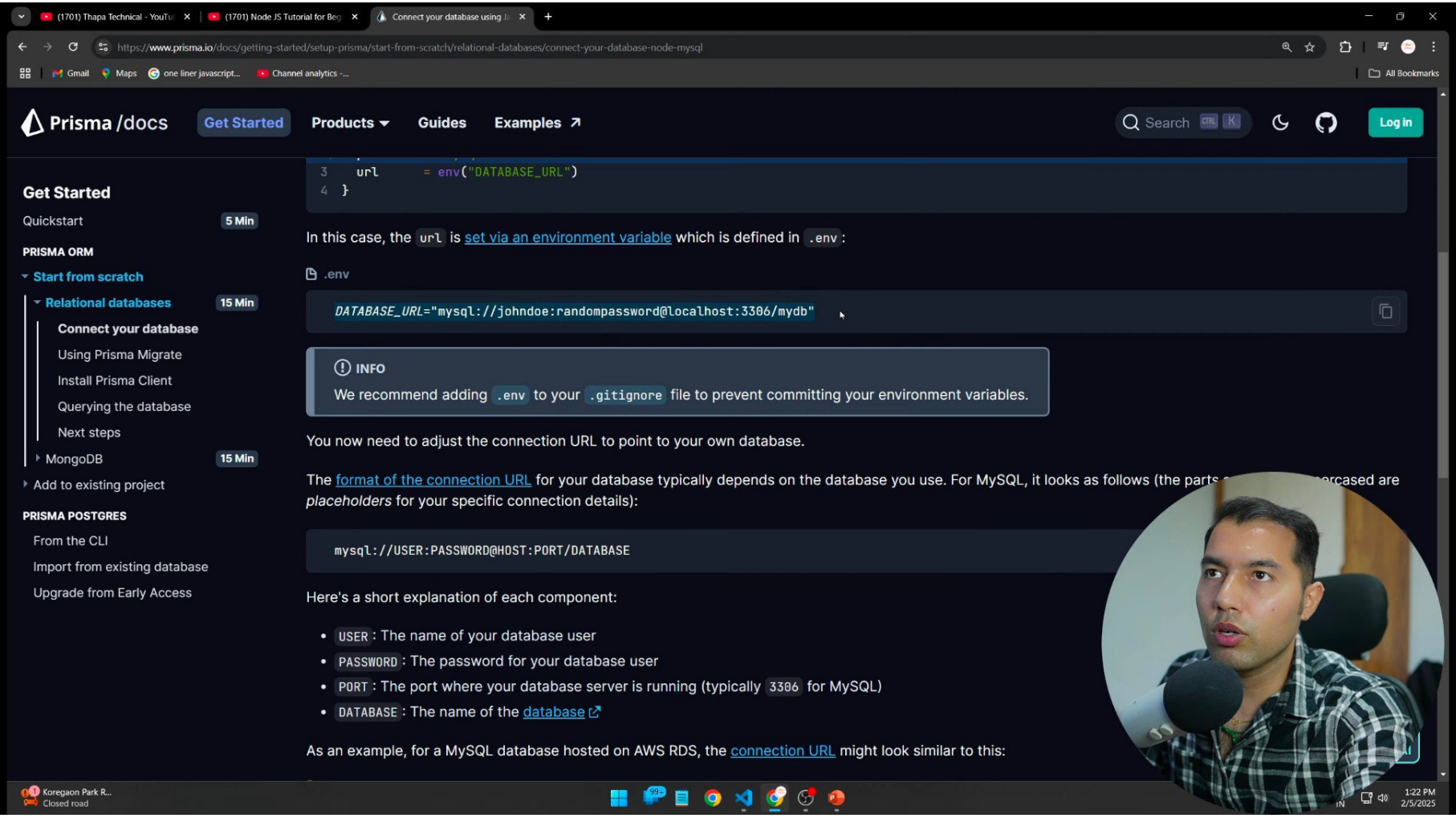
schema.prisma... | prisma_mysql - Visual...

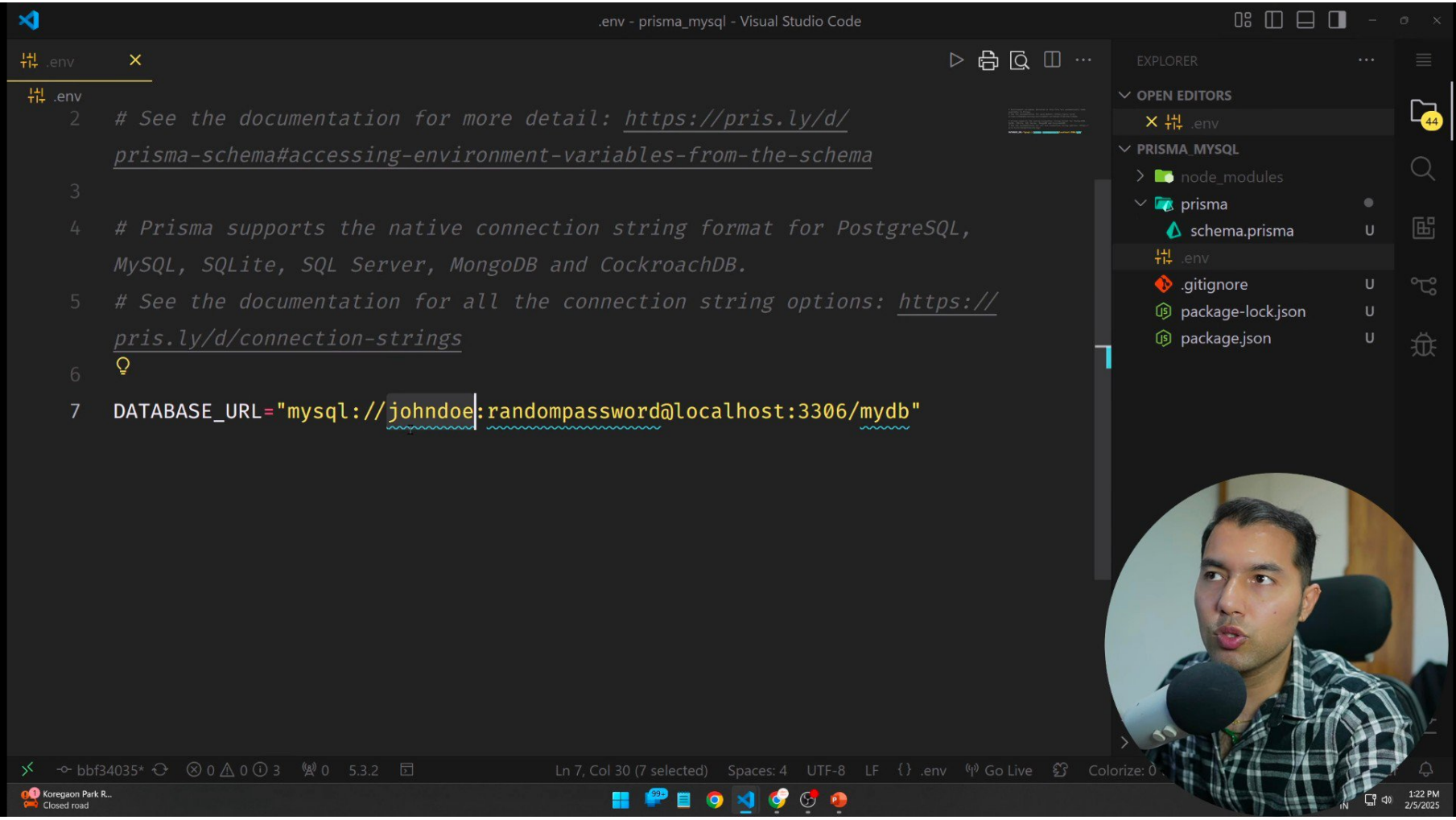


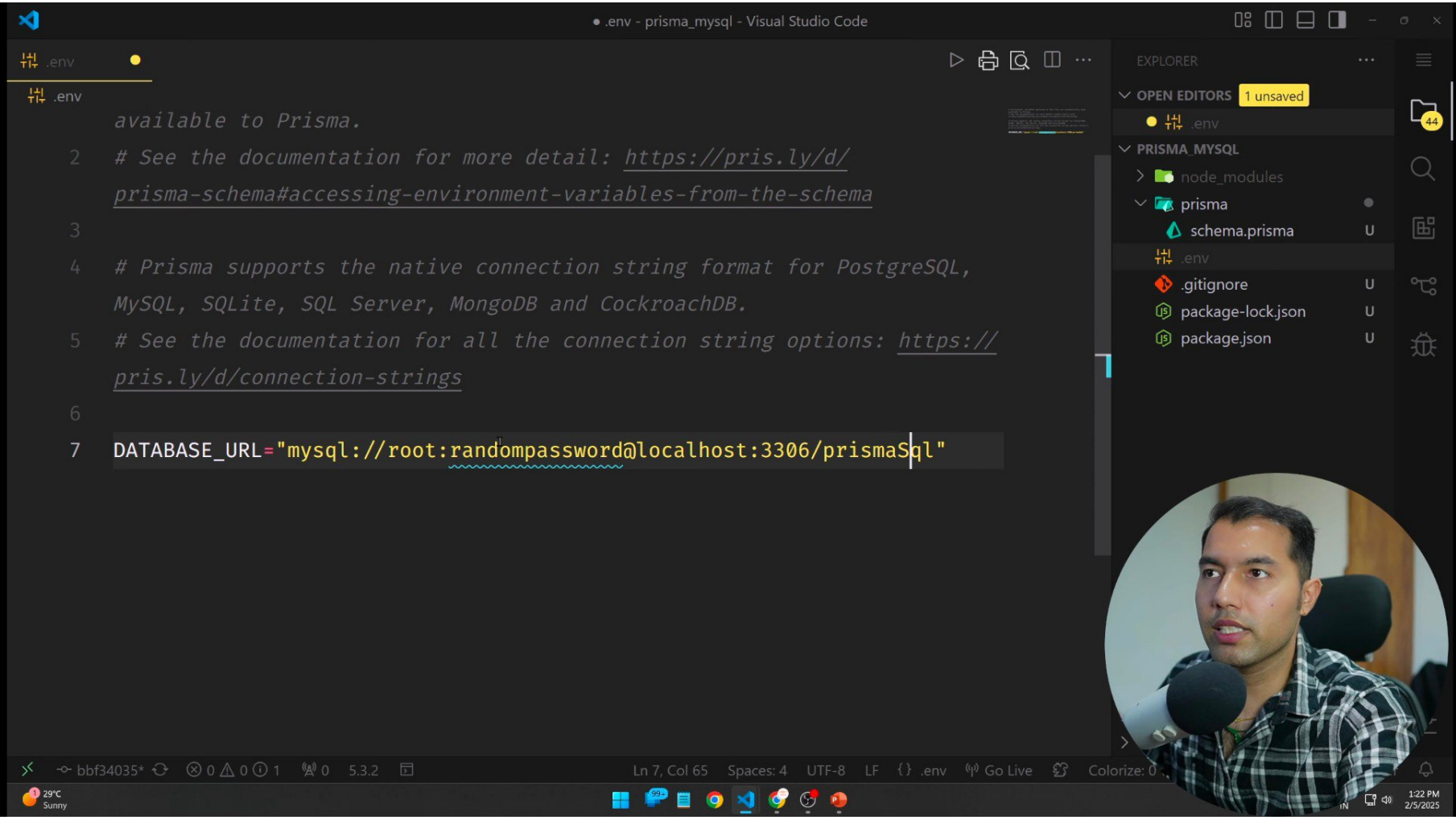
Finance headline
European shares...

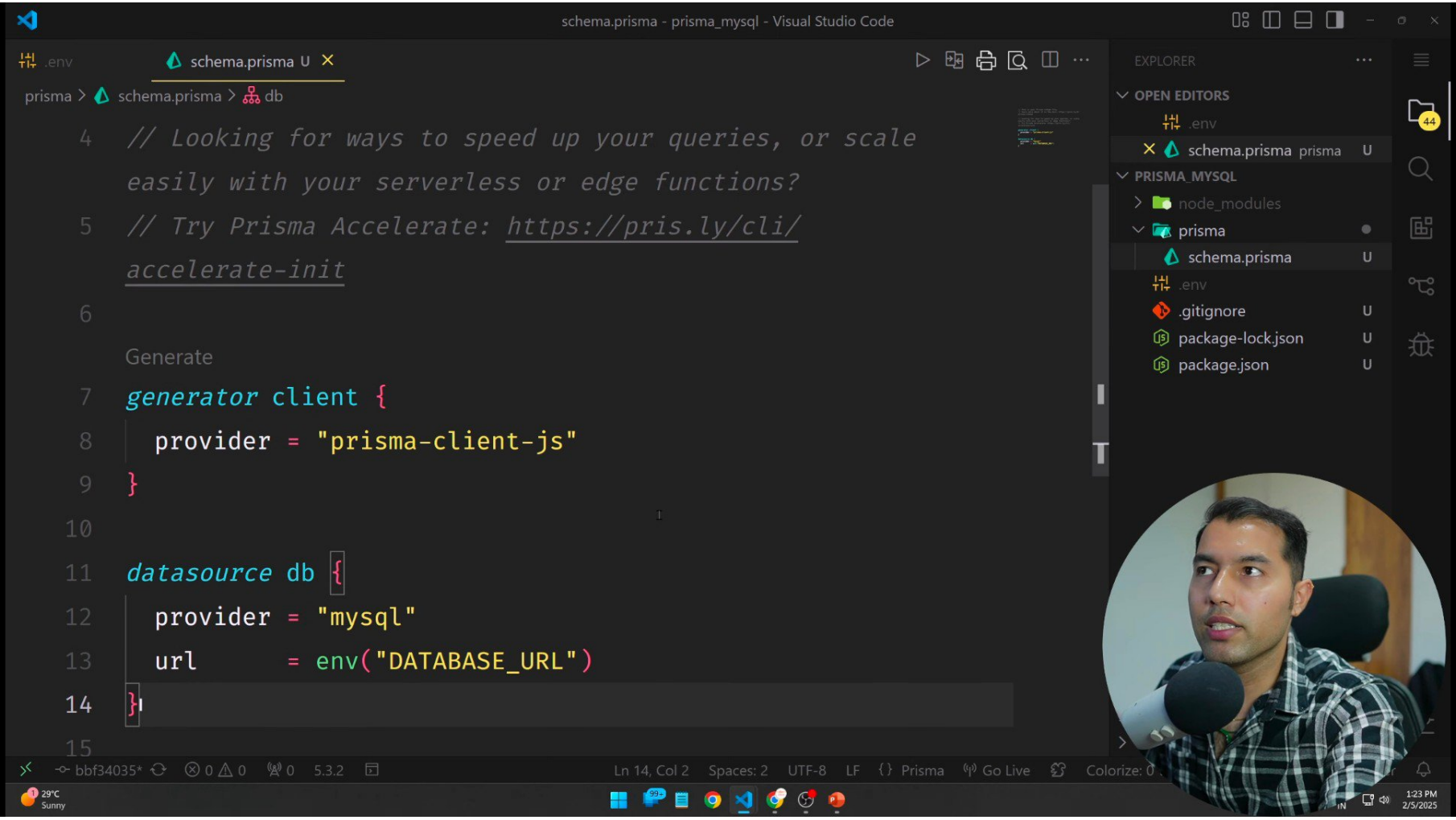
1:17 PM
2/5/2025

Thapa Technical


(1701) Thapa Technical - YouTu
(1701) Node JS Tutorial for Beg
Connect your database using
https://www.prisma.io/docs/getting-started/setup-prisma/start-from-scratch/relational-databases/connect-your-database-node-mysql
Gmail
Maps
one liner javascript...
Channel analytics -...
All Bookmarks
Prisma /docs
Get Started
Products
Guides
Examples
Search
Log in
Get Started
Quickstart
5 Min
PRISMA ORM
Start from scratch
Relational databases
15 Min
Connect your database
Using Prisma Migrate
Install Prisma Client
Querying the database
Next steps
MongoDB
15 Min
Add to existing project
PRISMA POSTGRES
From the CLI
Import from existing database
Upgrade from Early Access
url = env("DATABASE_URL")
}
In this case, the url is set via an environment variable which is defined in .env :
.env
DATABASE_URL="mysql://johndoe:randompassword@localhost:3306/mydb"
INFO
We recommend adding .env to your .gitignore file to prevent committing your environment variables.
You now need to adjust the connection URL to point to your own database.
The format of the connection URL for your database typically depends on the database you use. For MySQL, it looks as follows (the parts
placeholders for your specific connection details):
mysql://USER:PASSWORD@HOST:PORT/DATABASE
Here's a short explanation of each component:
USER : The name of your database user
PASSWORD : The password for your database user
PORT : The port where your database server is running (typically 3306 for MySQL)
DATABASE : The name of the database
As an example, for a MySQL database hosted on AWS RDS, the connection URL might look similar to this:
Koregaon Park R...
Closed road
1:22 PM
2/5/2025

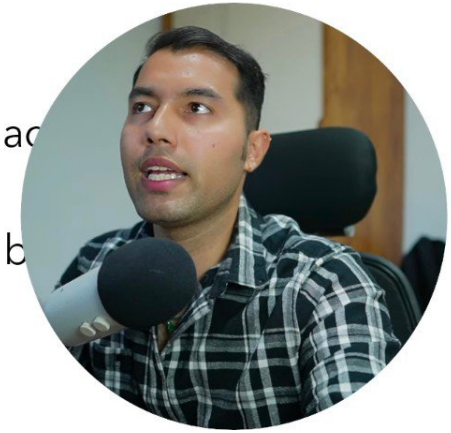

.env - prisma_mysql - Visual Studio Code
.env
# See the documentation for more detail: https://pris.ly/d/prisma-schema#accessing-environment-variables-from-the-schema
# Prisma supports the native connection string format for PostgreSQL, MySQL, SQLite, SQL Server, MongoDB and CockroachDB.
# See the documentation for all the connection string options: https://pris.ly/d/connection-strings
DATABASE_URL="mysql://johndoe:randompassword@localhost:3306/mydb"
EXPLORER
OPEN EDITORS
.env
PRISMA_MYSQL
node_modules
prisma
schema.prisma U
.env
.gitignore U
package-lock.json U
package.json U
Ln 7, Col 30 (7 selected)
Spaces: 4
UTF-8
LF
.env
Go Live
Koregaon Park R...
Closed road
1:22 PM
2/5/2025

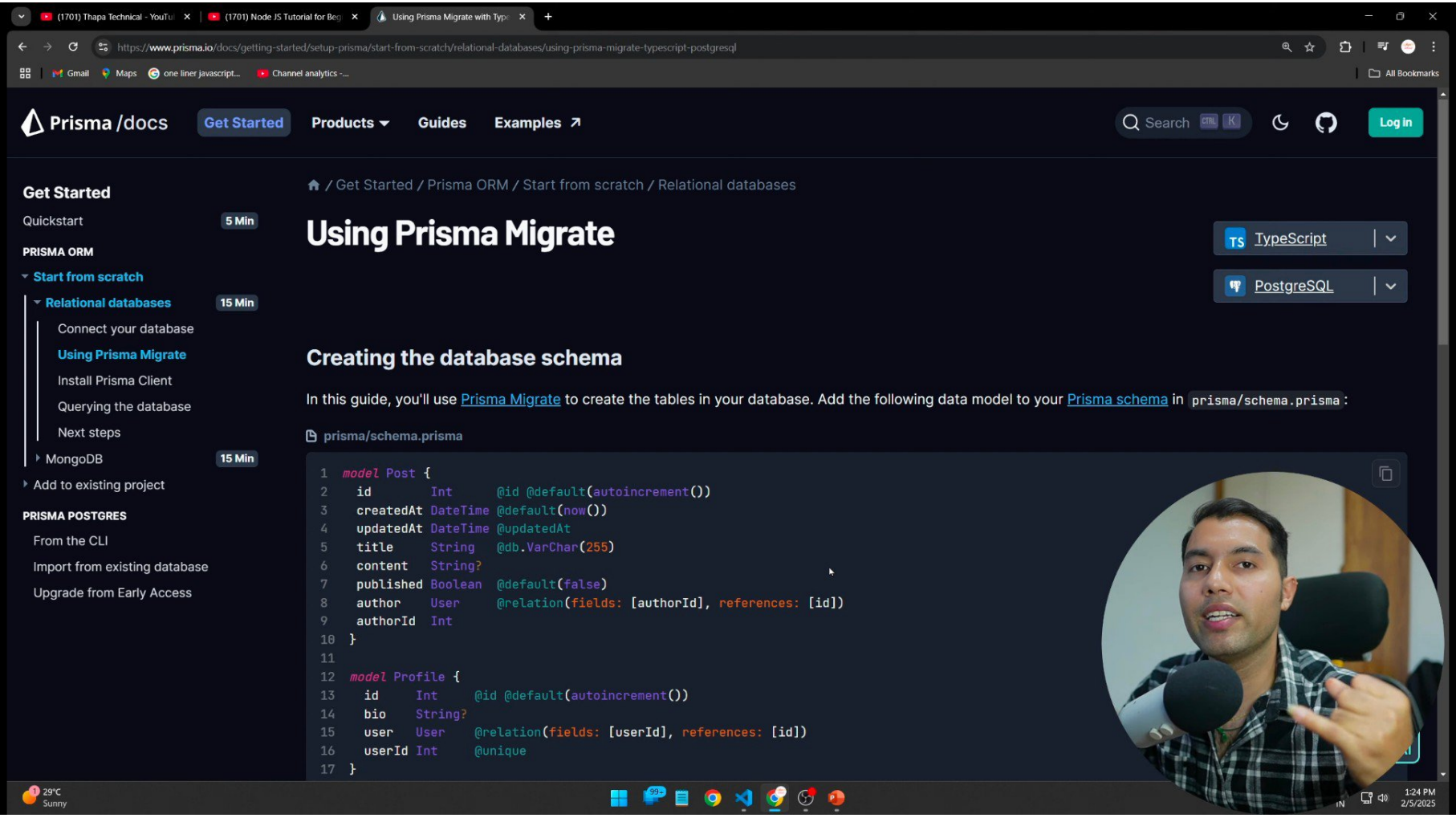

● .env - prisma_mysql - Visual Studio Code
.env
.env
available to Prisma.
# See the documentation for more detail: https://pris.ly/d/prisma-schema#accessing-environment-variables-from-the-schema
# Prisma supports the native connection string format for PostgreSQL, MySQL, SQLite, SQL Server, MongoDB and CockroachDB.
# See the documentation for all the connection string options: https://pris.ly/d/connection-strings
DATABASE_URL="mysql://root:randompassword@localhost:3306/prismaSql"
EXPLORER
OPEN EDITORS 1 unsaved
.env
PRISMA_MYSQL
node_modules
prisma
schema.prisma U
.env
.gitignore U
package-lock.json U
package.json U
bbf34035*
Ln 7, Col 65
Spaces: 4
UTF-8
LF
.env
Go Live
29°C
Sunny
1:22 PM
2/5/2025


schema.prisma - prisma_mysql - Visual Studio Code
.env
schema.prisma U
prisma > schema.prisma > db
// Looking for ways to speed up your queries, or scale easily with your serverless or edge functions?
// Try Prisma Accelerate: https://pris.ly/cli/accelerate-init
Generate
generator client {
provider = "prisma-client-js"
}
datasource db {
provider = "mysql"
url = env("DATABASE_URL")
}
EXPLORER
OPEN EDITORS
.env
schema.prisma prisma U
PRISMA_MYSQL
node_modules
prisma
schema.prisma U
.env
.gitignore U
package-lock.json U
package.json U
Ln 14, Col 2
Spaces: 2
UTF-8
LF
Prisma
Go Live
29°C
Sunny
1:23 PM
2/5/2025

Migrations in ORM

- As you know that, SQL based databases require you to assign schema which you can't do while backend is running; you have to do beforehand. Migrations is for that.
- Migration is the process of evolving and managing changes to a database schema over time.
- It involves creating, modifying, or deleting tables, columns, and relationships in the database.
- In Prisma, migrations are stored as versioned SQL script files.
- Migrations help ensure that database structures stay consistent ac environments (e.g., development, staging, production).
- Prisma Migrate generates and applies migrations automatically b changes in the Prisma schema.

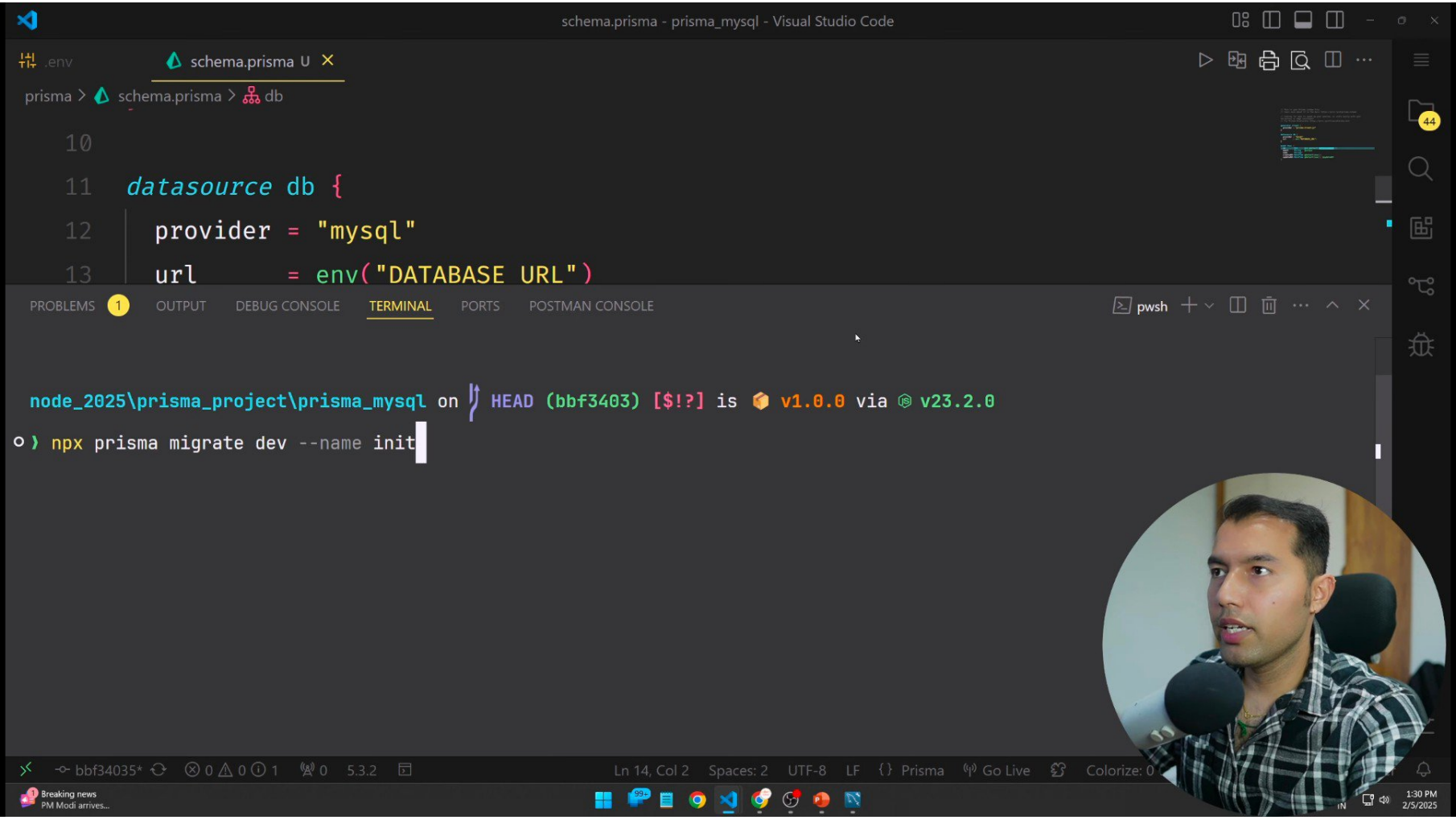



(1701) Thapa Technical - YouTu
(1701) Node JS Tutorial for Beg
Using Prisma Migrate with Type
https://www.prisma.io/docs/getting-started/setup-prisma/start-from-scratch/relational-databases/using-prisma-migrate-typescript-postgresql
Gmail
Maps
one liner javascript...
Channel analytics -...
All Bookmarks
Prisma /docs
Get Started
Products
Guides
Examples
Search
Log in
Get Started
Quickstart
5 Min
PRISMA ORM
Start from scratch
Relational databases
15 Min
Connect your database
Using Prisma Migrate
Install Prisma Client
Querying the database
Next steps
MongoDB
15 Min
Add to existing project
PRISMA POSTGRES
From the CLI
Import from existing database
Upgrade from Early Access
/ Get Started / Prisma ORM / Start from scratch / Relational databases
Using Prisma Migrate
TypeScript
PostgreSQL
Creating the database schema
In this guide, you'll use Prisma Migrate to create the tables in your database. Add the following data model to your Prisma schema in prisma/schema.prisma:
prisma/schema.prisma
model Post {
id Int @id @default(autoincrement())
createdAt DateTime @default(now())
updatedAt DateTime @updatedAt
title String @db.VarChar(255)
content String?
published Boolean @default(false)
author User @relation(fields: [authorId], references: [id])
authorId Int
}
model Profile {
id Int @id @default(autoincrement())
bio String?
user User @relation(fields: [userId], references: [id])
userId Int @unique
}
29°C
Sunny
1:24 PM
2/5/2025

In this guide, you'll use Prisma Migrate to create the tables in your database. Add the following data m

prisma/schema.prisma

```prisma
model Post {
  id        Int      @id @default(autoincrement())
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
  title     String   @db.VarChar(255)
  content   String?
  published Boolean  @default(false)
  author    User     @relation(fields: [authorId], references: [id])
  authorId  Int
}

model Profile {
  id     Int     @id @default(autoincrement())
  bio    String?
  user   User    @relation(fields: [userId], references: [id])
  userId Int     @unique
}
```

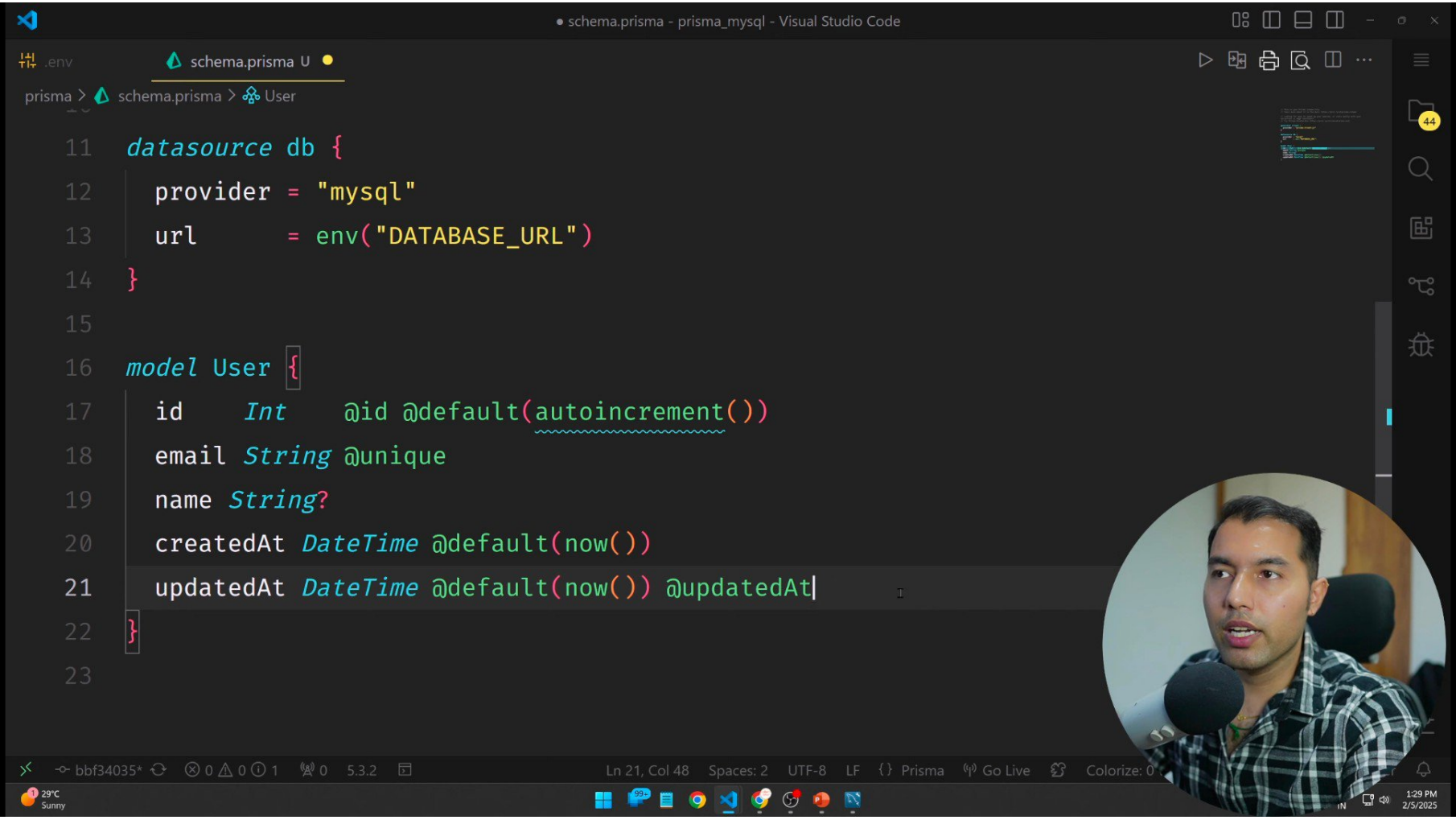

schema.prisma - prisma_mysql - Visual Studio Code
.env
schema.prisma U
prisma > schema.prisma > db
10
11 datasource db {
12 provider = "mysql"
13 url = env("DATABASE URL")
PROBLEMS 1
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
POSTMAN CONSOLE
pwsh
node_2025\prisma_project\prisma_mysql on HEAD (bbf3403) [$!?] is v1.0.0 via v23.2.0
npx prisma migrate dev --name init
bbf34035*
Ln 14, Col 2
Spaces: 2
UTF-8
LF
Prisma
Go Live
Colorize: 0
Breaking news
PM Modi arrives...
1:30 PM
2/5/2025

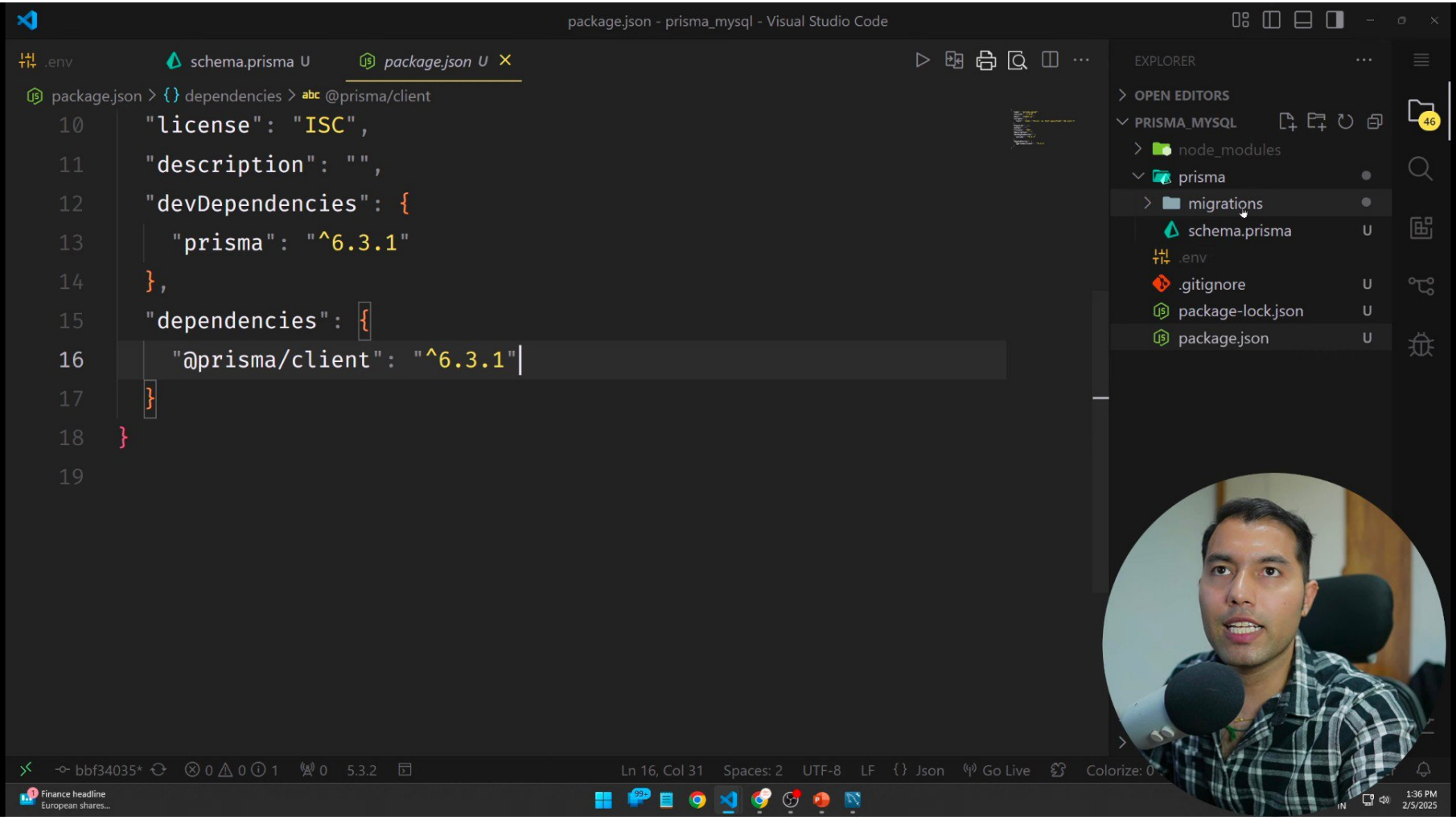
Create ShortLink model

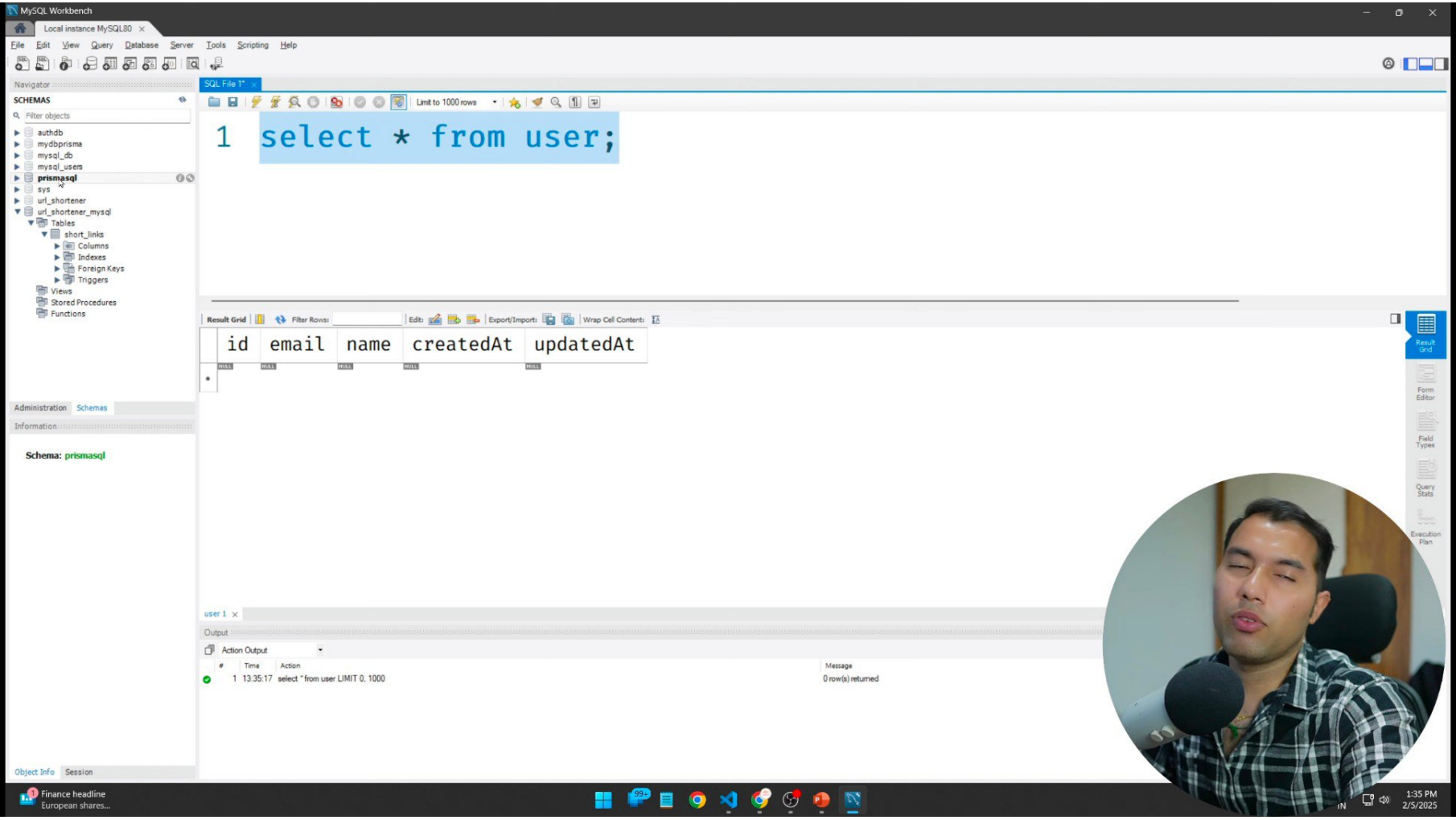
```
model ShortLink {
  id          Int    @id @default(autoincrement())
  shortCode   String @unique
  url         String
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
}
```

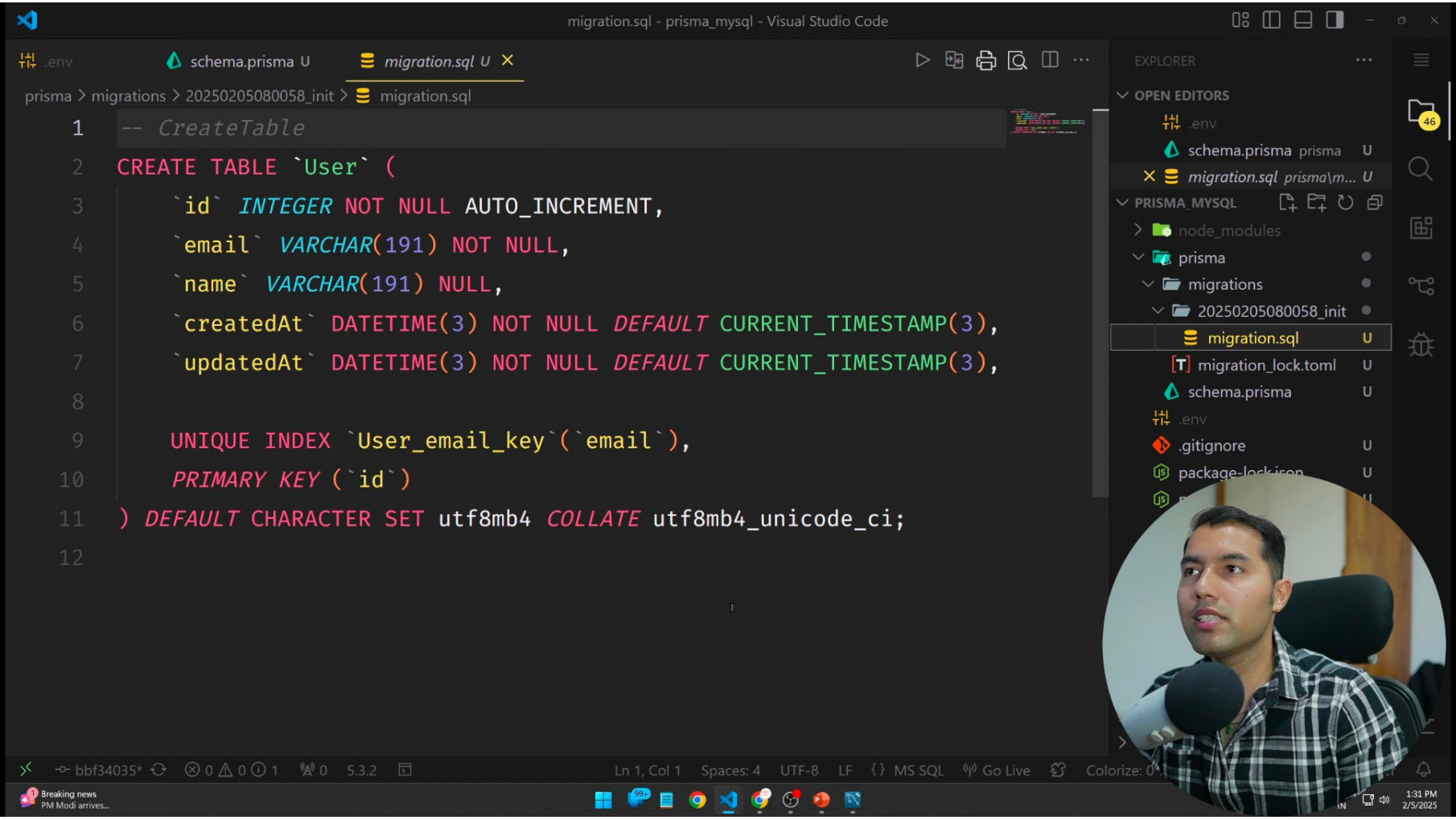
Migrate it

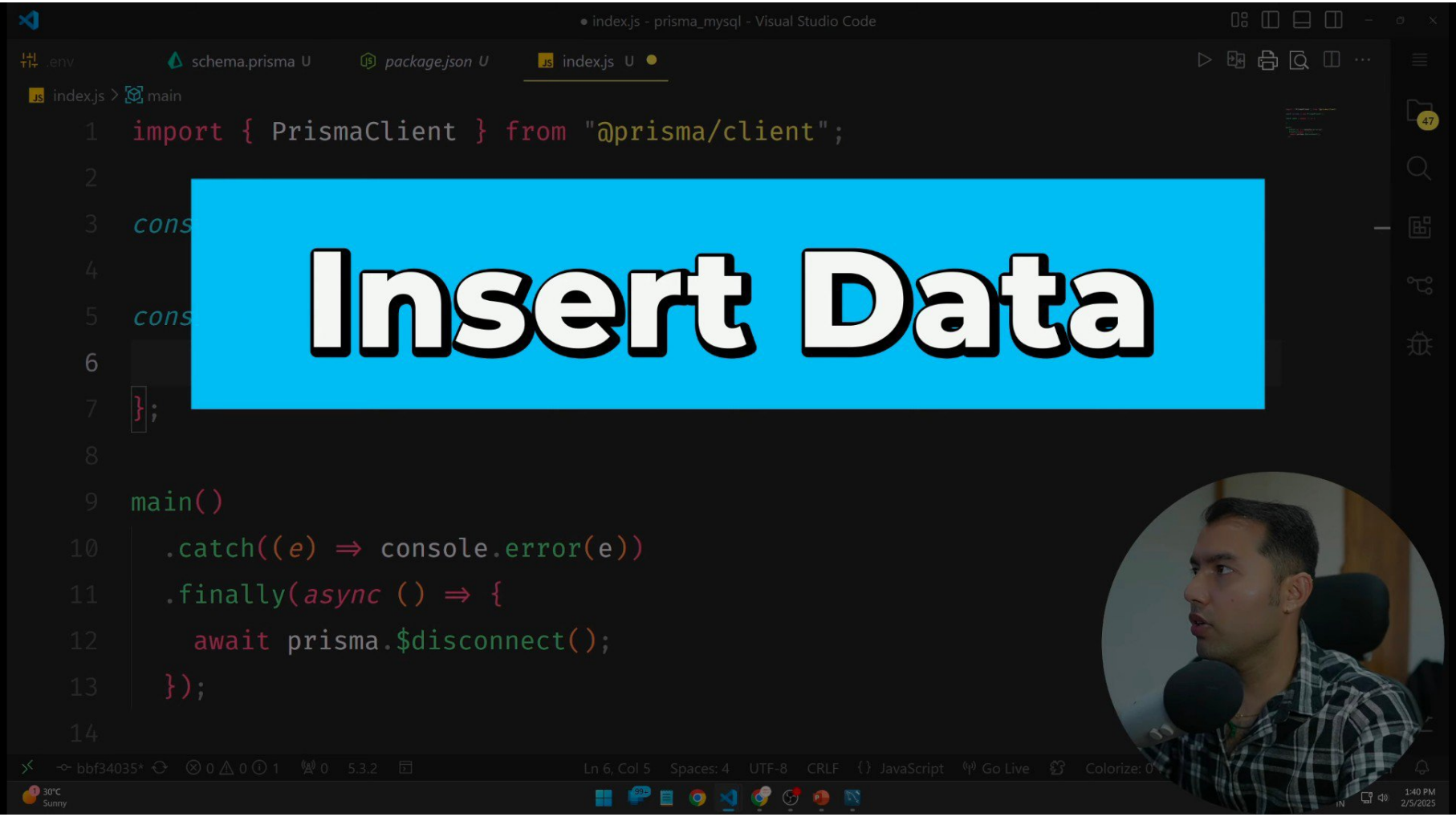
- Now, run **npx prisma migrate dev** to create migration files and push it.
- You can pass **--name** flag with it to include name:
  - **npx prisma migrate dev --name init**
- In Production, you can run **npx prisma migrate deploy** to run migration files.
- You can also include both of them in **package.json** scripts

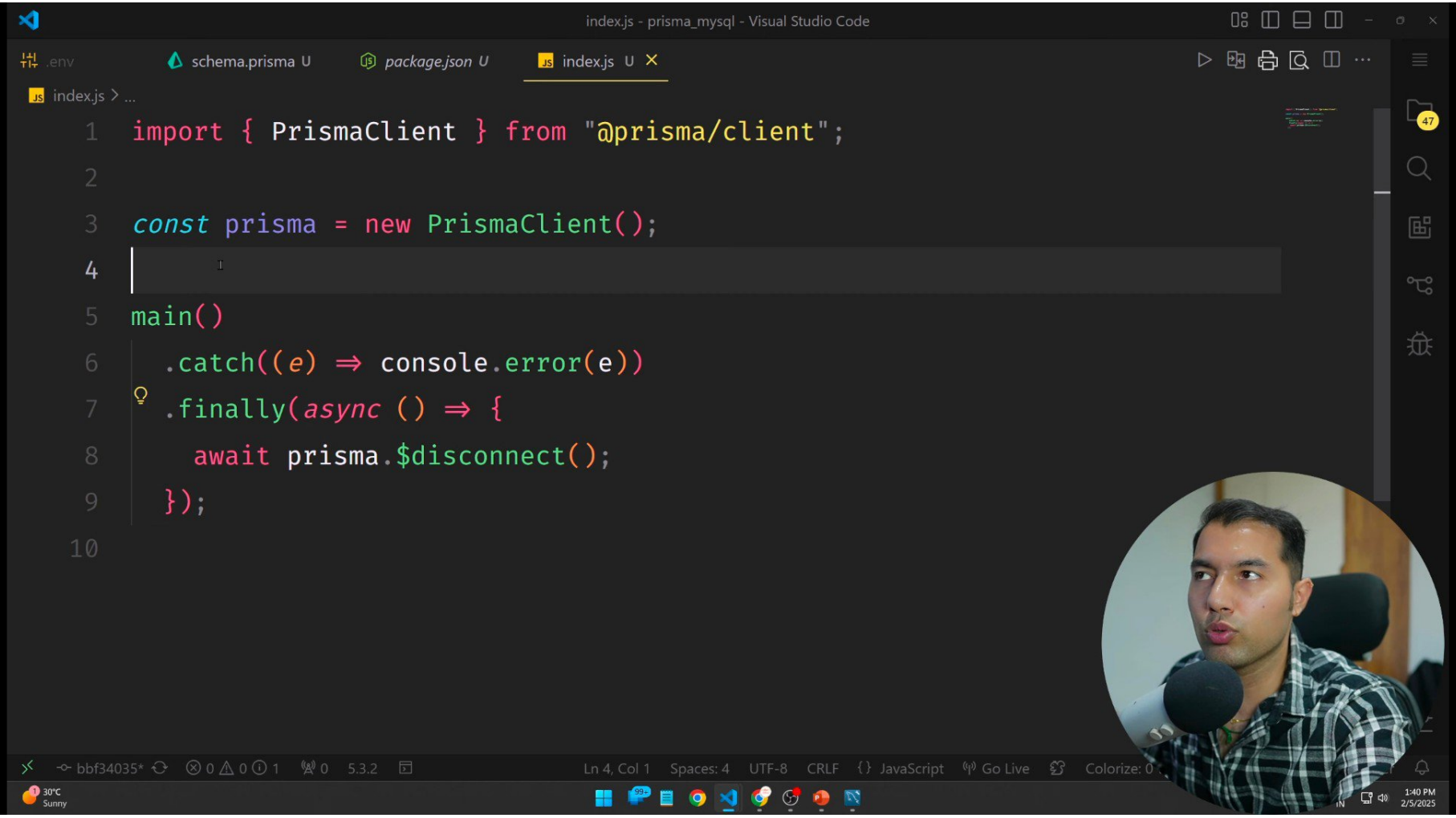

schema.prisma - prisma_mysql - Visual Studio Code
.env
schema.prisma U
prisma > schema.prisma > User
11 datasource db {
12 provider = "mysql"
13 url = env("DATABASE_URL")
14 }
15
16 model User {
17 id Int @id @default(autoincrement())
18 email String @unique
19 name String?
20 createdAt DateTime @default(now())
21 updatedAt DateTime @default(now()) @updatedAt
22 }
23
Ln 21, Col 48 Spaces: 2 UTF-8 LF Prisma Go Live
29°C Sunny
1:29 PM 2/5/2025

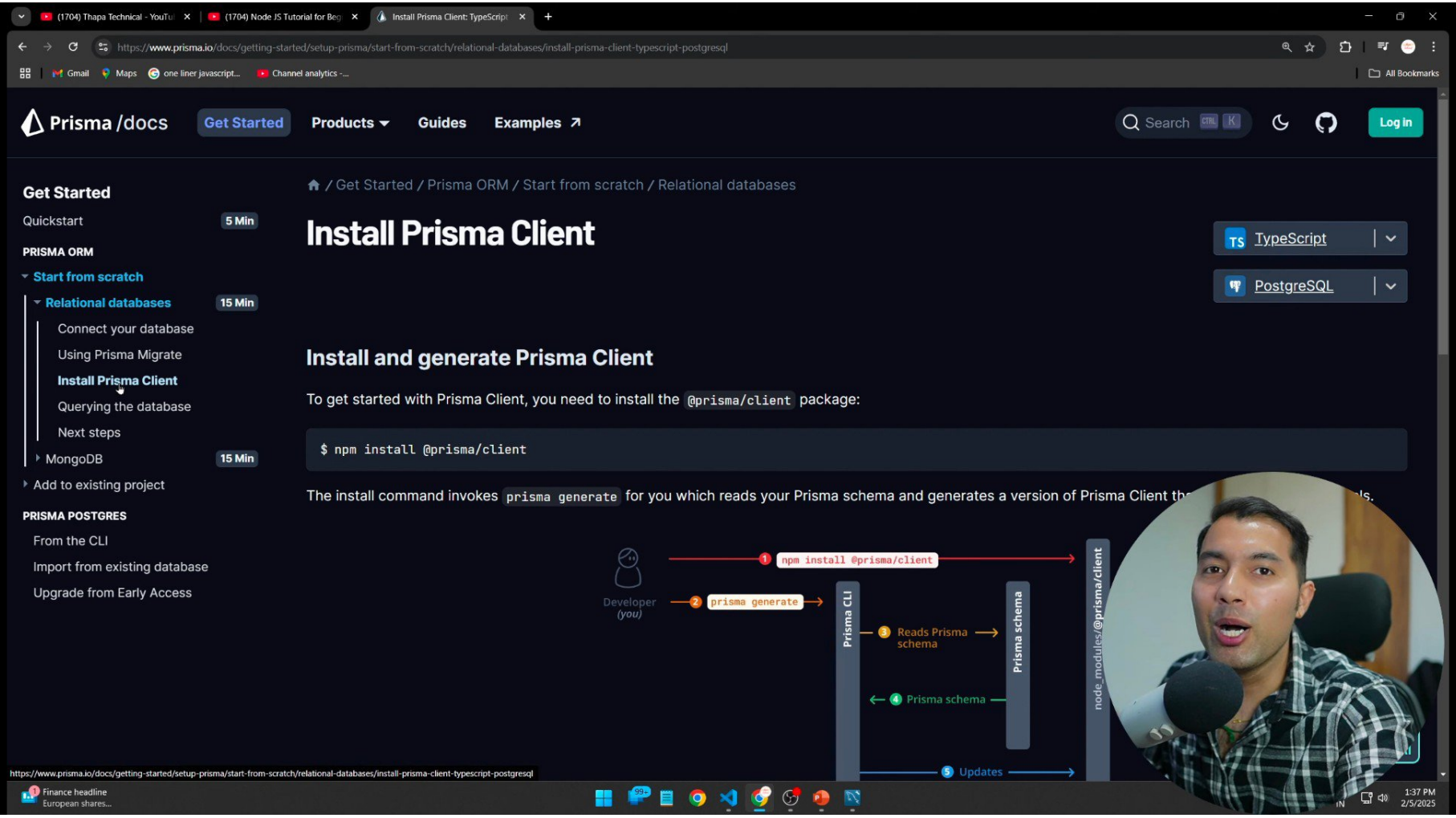

package.json - prisma_mysql - Visual Studio Code
.env
schema.prisma U
package.json U
package.json > { } dependencies > abc @prisma/client
10 "license": "ISC",
11 "description": "",
12 "devDependencies": {
13 "prisma": "^6.3.1"
14 },
15 "dependencies": {
16 "@prisma/client": "^6.3.1"
17 }
18 }
19
EXPLORER
OPEN EDITORS
PRISMA_MYSQL
node_modules
prisma
migrations
schema.prisma U
.env
.gitignore U
package-lock.json U
package.json U
bbf34035*
Ln 16, Col 31
Spaces: 2
UTF-8
LF
{ } Json
Go Live
Finance headline
European shares...
1:36 PM
2/5/2025


MySQL Workbench
Local instance MySQL80
File Edit View Query Database Server Tools Scripting Help
Navigator
SCHEMAS
Filter objects
authdb
mydbprisma
mysql_db
mysql_users
prismasql
sys
url_shortener
url_shortener_mysql
Tables
short_links
Columns
Indexes
Foreign Keys
Triggers
Views
Stored Procedures
Functions
Administration Schemas
Information
Schema: prismasql
Object Info Session
SQL File 1*
Limit to 1000 rows
1 select * from user;
Result Grid Filter Rows: Edit: Export/Import: Wrap Cell Content:
id email name createdAt updatedAt
user 1
Output
Action Output
Time Action Message
1 13:35:17 select * from user LIMIT 0, 1000 0 row(s) returned
Finance headline
European shares...
1:35 PM
2/5/2025


migration.sql - prisma_mysql - Visual Studio Code
.env
schema.prisma U
migration.sql U
prisma > migrations > 20250205080058_init > migration.sql
-- CreateTable
CREATE TABLE `User` (
`id` INTEGER NOT NULL AUTO_INCREMENT,
`email` VARCHAR(191) NOT NULL,
`name` VARCHAR(191) NULL,
`createdAt` DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
`updatedAt` DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
UNIQUE INDEX `User_email_key`(`email`),
PRIMARY KEY (`id`)
) DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
EXPLORER
OPEN EDITORS
schema.prisma prisma U
migration.sql prisma\m... U
PRISMA_MYSQL
node_modules
prisma
migrations
20250205080058_init
migration.sql U
migration_lock.toml U
schema.prisma U
.gitignore U
package-lock.json U
Ln 1, Col 1 Spaces: 4 UTF-8 LF MS SQL Go Live
Breaking news
PM Modi arrives...
1:31 PM
2/5/2025


index.js - prisma_mysql - Visual Studio Code
.env
schema.prisma U
package.json U
index.js U
index.js > main
import { PrismaClient } from "@prisma/client";
Insert Data
};
main()
.catch((e) => console.error(e))
.finally(async () => {
await prisma.$disconnect();
});
Ln 6, Col 5
Spaces: 4
UTF-8
CRLF
JavaScript
Go Live
30°C
Sunny
1:40 PM
2/5/2025


index.js - prisma_mysql - Visual Studio Code
.env
schema.prisma U
package.json U
index.js U
index.js > ...
1 import { PrismaClient } from "@prisma/client";
2
3 const prisma = new PrismaClient();
4
5 main()
6 .catch((e) => console.error(e))
7 .finally(async () => {
8 await prisma.$disconnect();
9 });
10
bbf34035*
Ln 4, Col 1
Spaces: 4
UTF-8
CRLF
JavaScript
Go Live
30°C
Sunny
1:40 PM
2/5/2025


(1704) Thapa Technical - YouTu
(1704) Node JS Tutorial for Beg
Install Prisma Client: TypeScript
https://www.prisma.io/docs/getting-started/setup-prisma/start-from-scratch/relational-databases/install-prisma-client-typescript-postgresql
Gmail
Maps
one liner javascript...
Channel analytics -...
All Bookmarks
Prisma /docs
Get Started
Products
Guides
Examples
Search
Log in
Get Started
Quickstart
5 Min
PRISMA ORM
Start from scratch
Relational databases
15 Min
Connect your database
Using Prisma Migrate
Install Prisma Client
Querying the database
Next steps
MongoDB
15 Min
Add to existing project
PRISMA POSTGRES
From the CLI
Import from existing database
Upgrade from Early Access
/ Get Started / Prisma ORM / Start from scratch / Relational databases
Install Prisma Client
TypeScript
PostgreSQL
Install and generate Prisma Client
To get started with Prisma Client, you need to install the @prisma/client package:
$ npm install @prisma/client
The install command invokes prisma generate for you which reads your Prisma schema and generates a version of Prisma Client th
ls.
Developer
(you)
1 npm install @prisma/client
2 prisma generate
Prisma CLI
3 Reads Prisma schema
Prisma schema
4 Prisma schema
node_modules/@prisma/client
5 Updates
https://www.prisma.io/docs/getting-started/setup-prisma/start-from-scratch/relational-databases/install-prisma-client-typescript-postgresql
Finance headline
European shares...
1:37 PM
2/5/2025

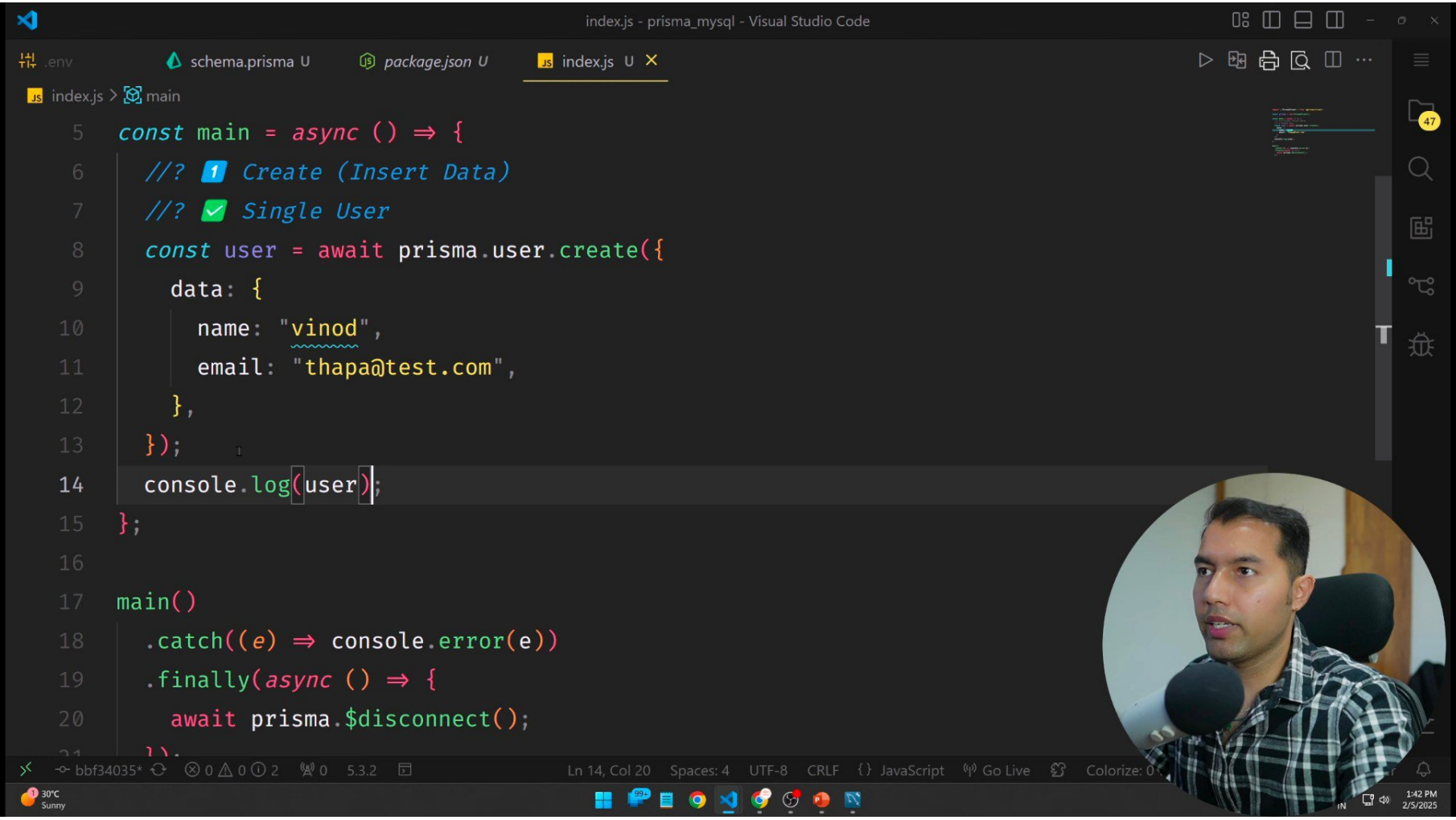
Prisma Client

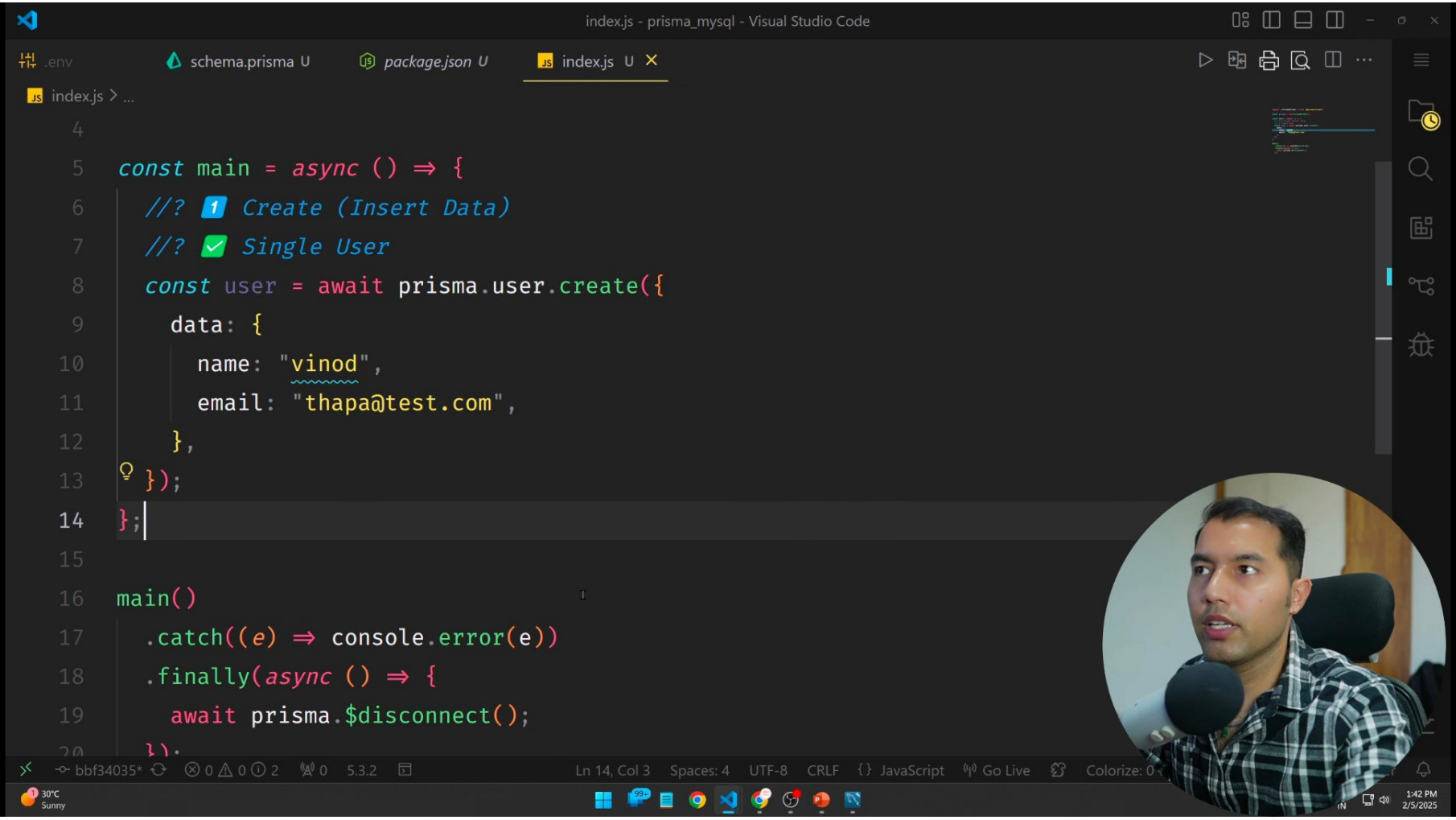
```
import { PrismaClient } from "@prisma/client";

export const prisma = new PrismaClient();

await prisma.shortLink.findMany();
```

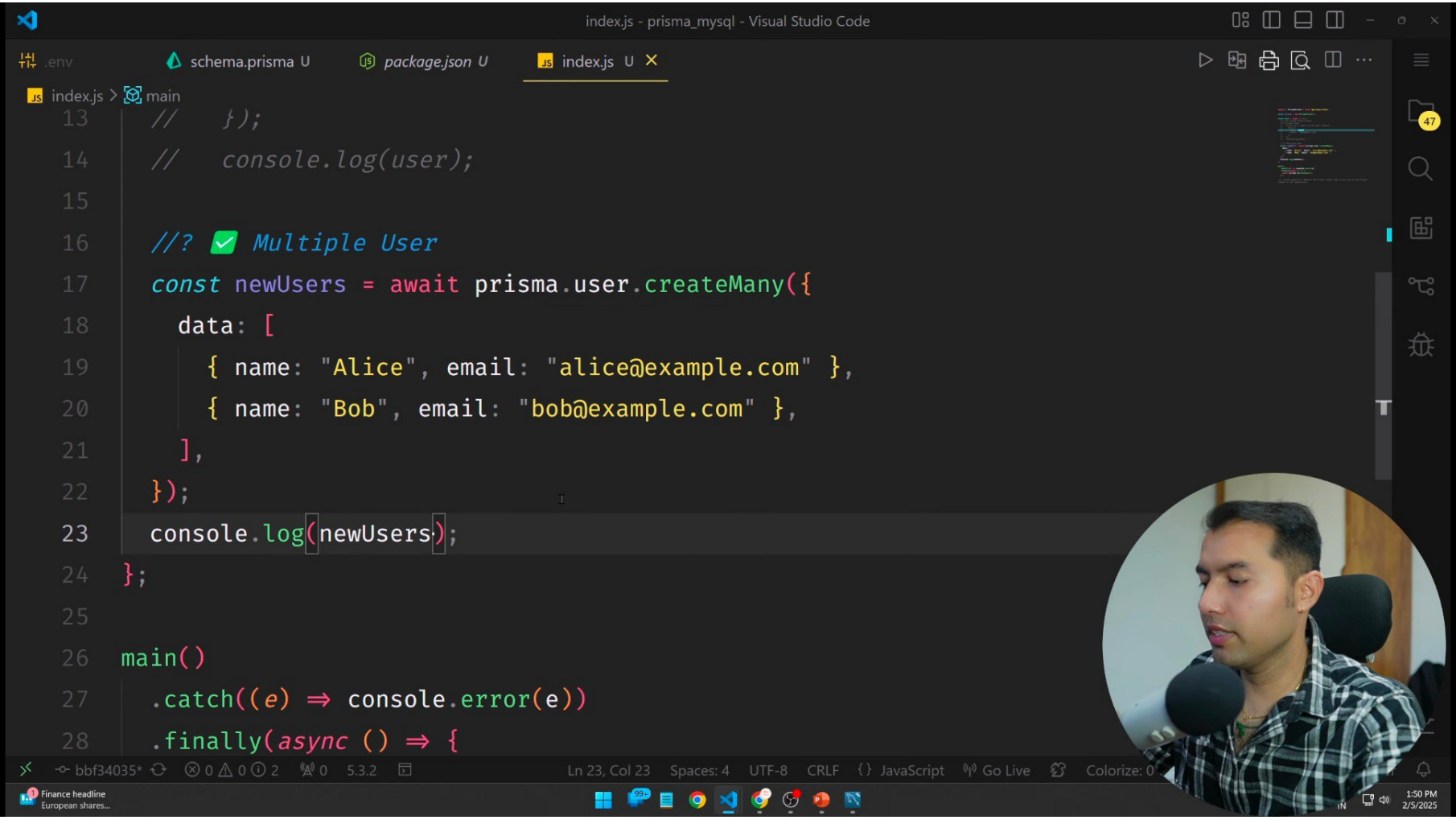
- Install Prisma Client
  - **npm install @prisma/client**
- Add this in **schema.prisma** above models to enable Prisma Client generation:

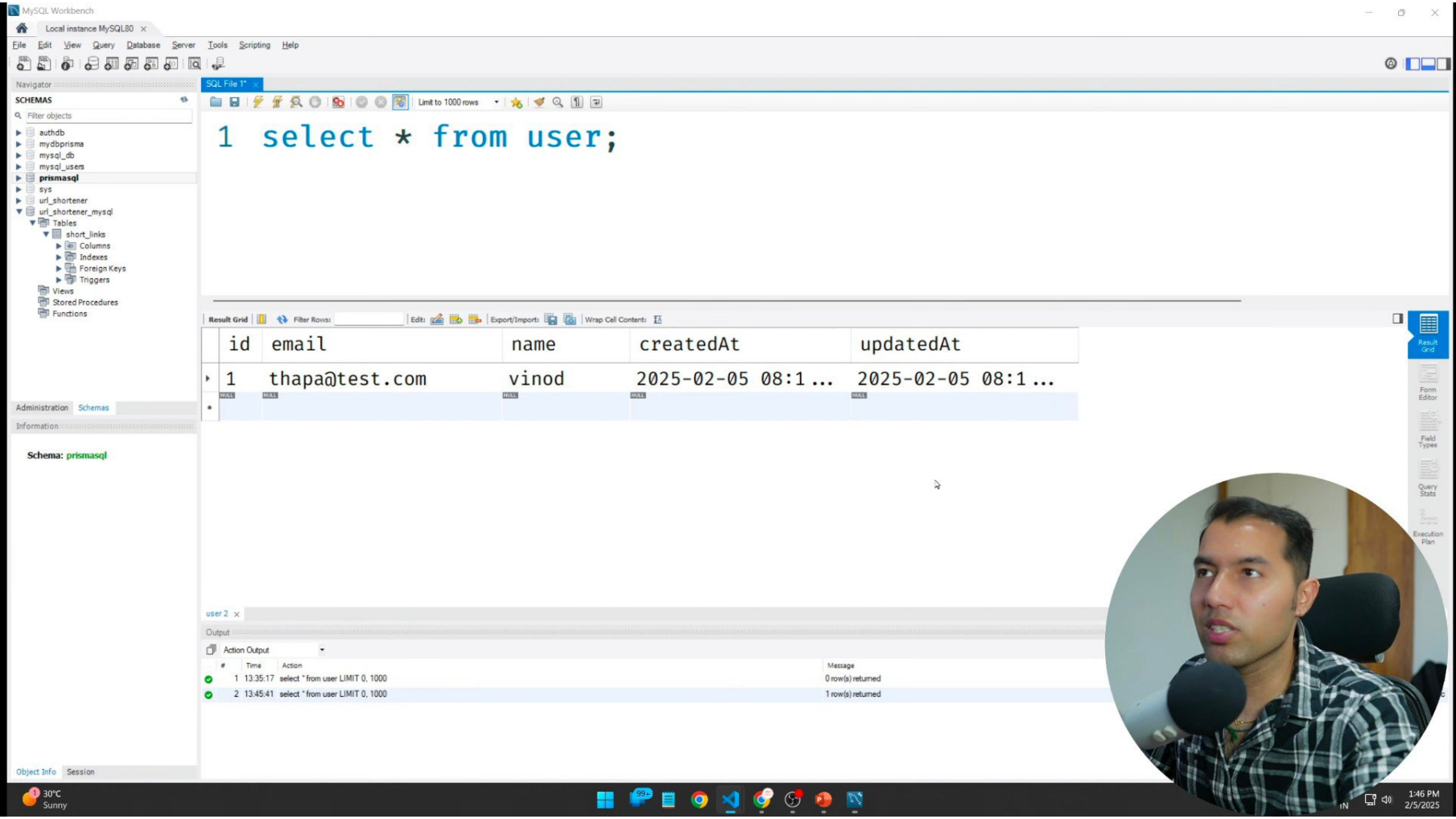
```
generator client {
  provider = "prisma-client-js“
}
```

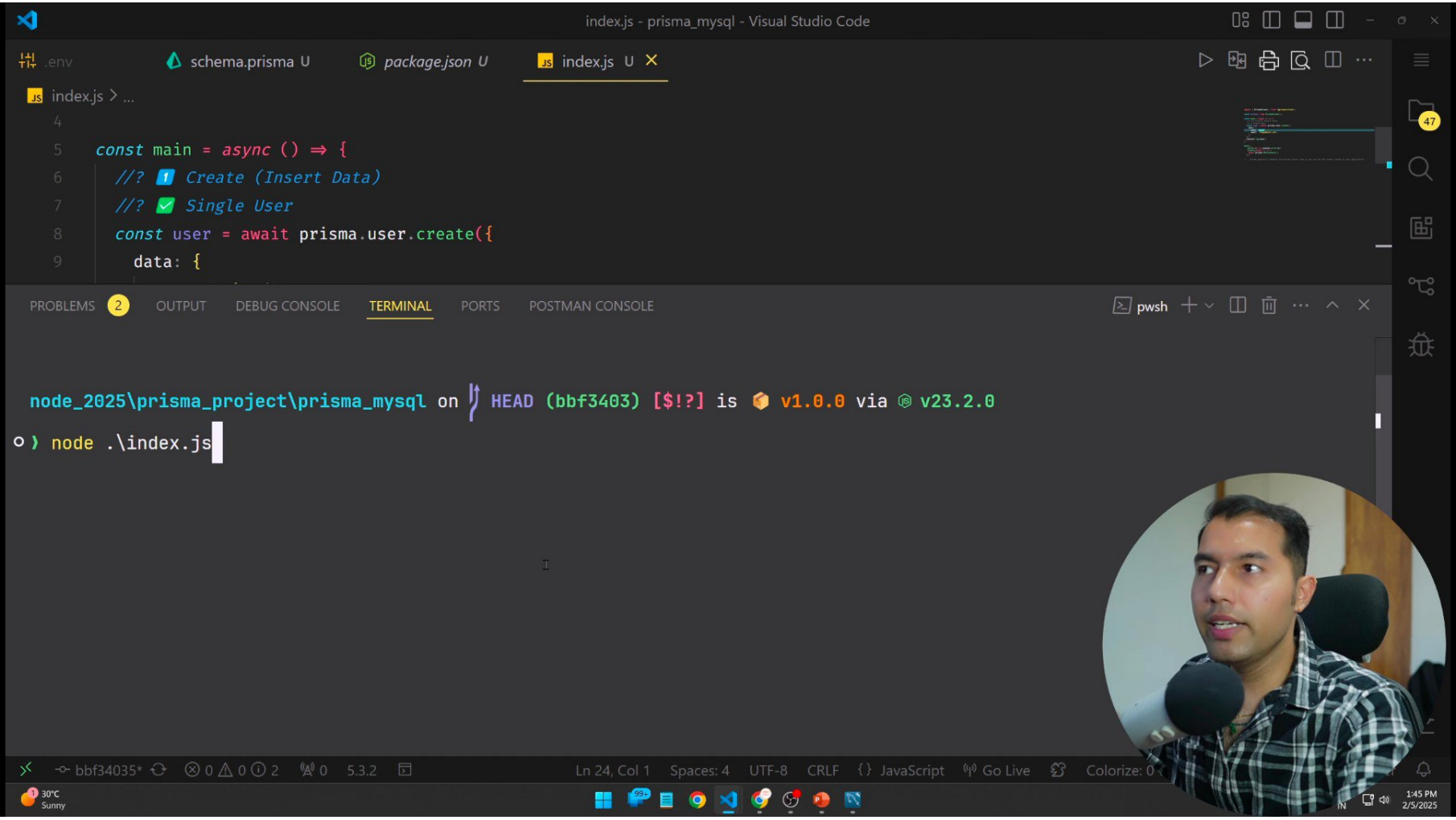
- Run **npx prisma generate** to generate Prisma Client after defini
- **npx prisma generate** is automatically executed when running **n migrate dev**.
- It can also be run manually if the schema is updated outside of m


index.js - prisma_mysql - Visual Studio Code
.env
schema.prisma U
package.json U
index.js U
index.js > main
const main = async () => {
//? 1 Create (Insert Data)
//? ✅ Single User
const user = await prisma.user.create({
data: {
name: "vinod",
email: "thapa@test.com",
},
});
console.log(user);
};
main()
.catch((e) => console.error(e))
.finally(async () => {
await prisma.$disconnect();
Ln 14, Col 20
Spaces: 4
UTF-8
CRLF
JavaScript
Go Live
30°C
Sunny
1:42 PM
2/5/2025


index.js - prisma_mysql - Visual Studio Code
.env
schema.prisma U
package.json U
index.js U
index.js > ...
const main = async () => {
//? 1 Create (Insert Data)
//? ✅ Single User
const user = await prisma.user.create({
data: {
name: "vinod",
email: "thapa@test.com",
},
});
};
main()
.catch((e) => console.error(e))
.finally(async () => {
await prisma.$disconnect();
});
bbf34035* 0 0 2 0 5.3.2
Ln 14, Col 3 Spaces: 4 UTF-8 CRLF JavaScript Go Live
30°C Sunny
1:42 PM 2/5/2025


index.js - prisma_mysql - Visual Studio Code
.env
schema.prisma U
package.json U
index.js U
index.js > main
// });
// console.log(user);
//? ✅ Multiple User
const newUsers = await prisma.user.createMany({
data: [
{ name: "Alice", email: "alice@example.com" },
{ name: "Bob", email: "bob@example.com" },
],
});
console.log(newUsers);
};
main()
.catch((e) => console.error(e))
.finally(async () => {
Ln 23, Col 23
Spaces: 4
UTF-8
CRLF
JavaScript
Go Live
Finance headline
European shares...
1:50 PM
2/5/2025


MySQL Workbench
Local instance MySQL80
File Edit View Query Database Server Tools Scripting Help
Navigator
SCHEMAS
Filter objects
authdb
mydbprisma
mysql_db
mysql_users
prismasql
sys
url_shortener
url_shortener_mysql
Tables
short_links
Columns
Indexes
Foreign Keys
Triggers
Views
Stored Procedures
Functions
Administration Schemas
Information
Schema: prismasql
Object Info Session
SQL File 1*
Limit to 1000 rows
1 select * from user;
Result Grid Filter Rows: Edit: Export/Import: Wrap Cell Content:
id email name createdAt updatedAt
1 thapa@test.com vinod 2025-02-05 08:1... 2025-02-05 08:1...
user 2
Output
Action Output
Time Action Message
1 13:35:17 select * from user LIMIT 0, 1000 0 row(s) returned
2 13:45:41 select * from user LIMIT 0, 1000 1 row(s) returned
30°C Sunny
1:46 PM 2/5/2025

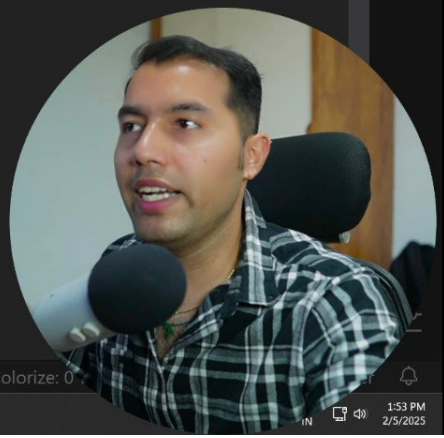

index.js - prisma_mysql - Visual Studio Code
.env
schema.prisma U
package.json U
index.js U
index.js > ...
const main = async () => {
//? 1 Create (Insert Data)
//? ✅ Single User
const user = await prisma.user.create({
data: {
PROBLEMS 2
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
POSTMAN CONSOLE
pwsh
node_2025\prisma_project\prisma_mysql on HEAD (bbf3403) [$!?] is v1.0.0 via v23.2.0
node .\index.js
bbf34035*
Ln 24, Col 1
Spaces: 4
UTF-8
CRLF
JavaScript
Go Live
30°C
Sunny
1:45 PM
2/5/2025

index.js - prisma_mysql - Visual Studio Code

.env | schema.prisma U | package.json U | index.js U

index.js > main > user

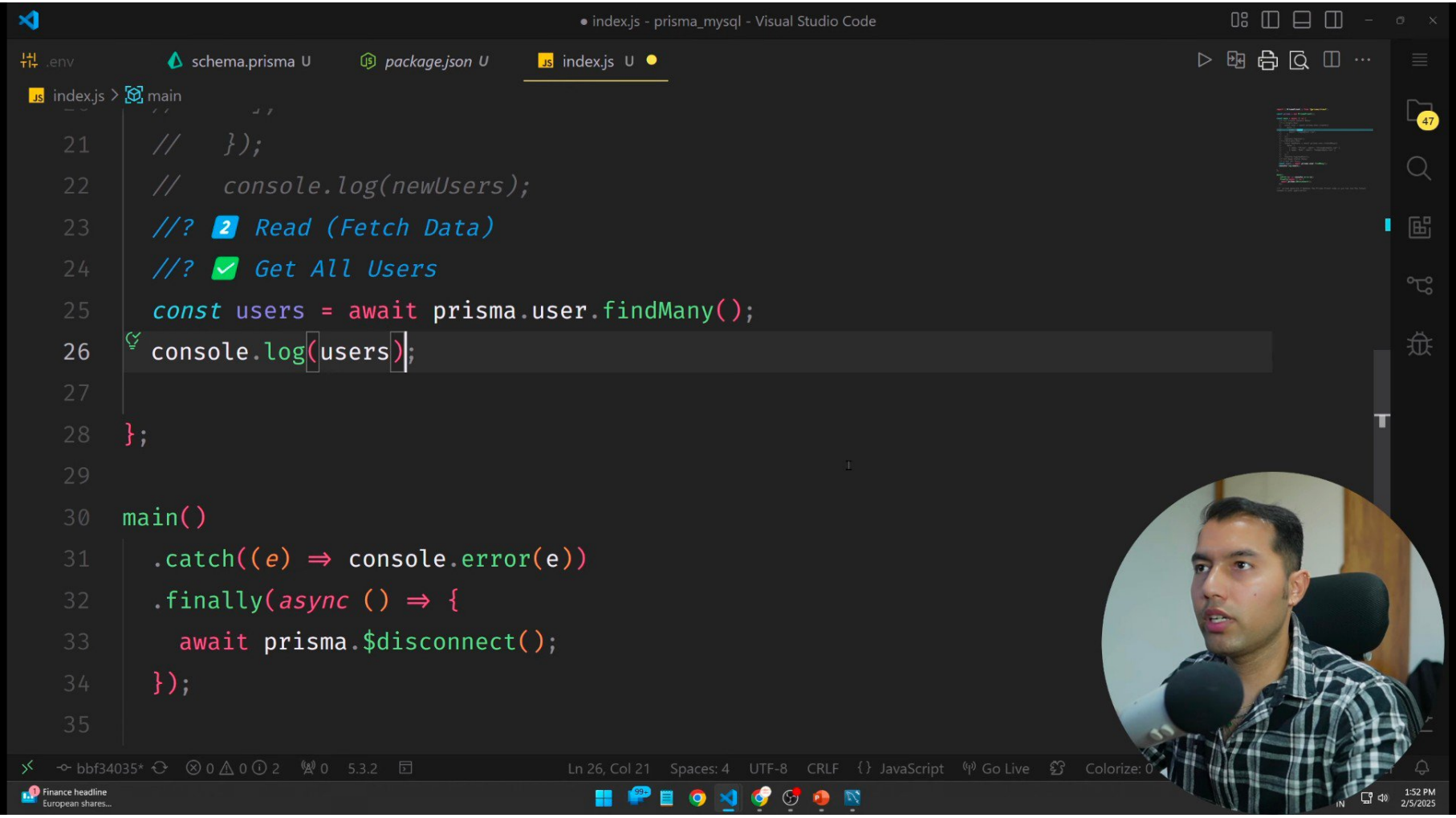
```
23    //? 2️⃣ Read (Fetch Data)
24    //? ✅ Get All Users
25    //   const users = await prisma.user.findMany();
26    //   console.log(users);
27    //?   ✅ Get a Single User by ID
28    const user = await prisma.user.findUnique({
29      where: { id: 2 },
30    });
31    console.log(users);
32  };
33
34  main()
35    .catch((e) => console.error(e))
36    .finally(async () => {
```



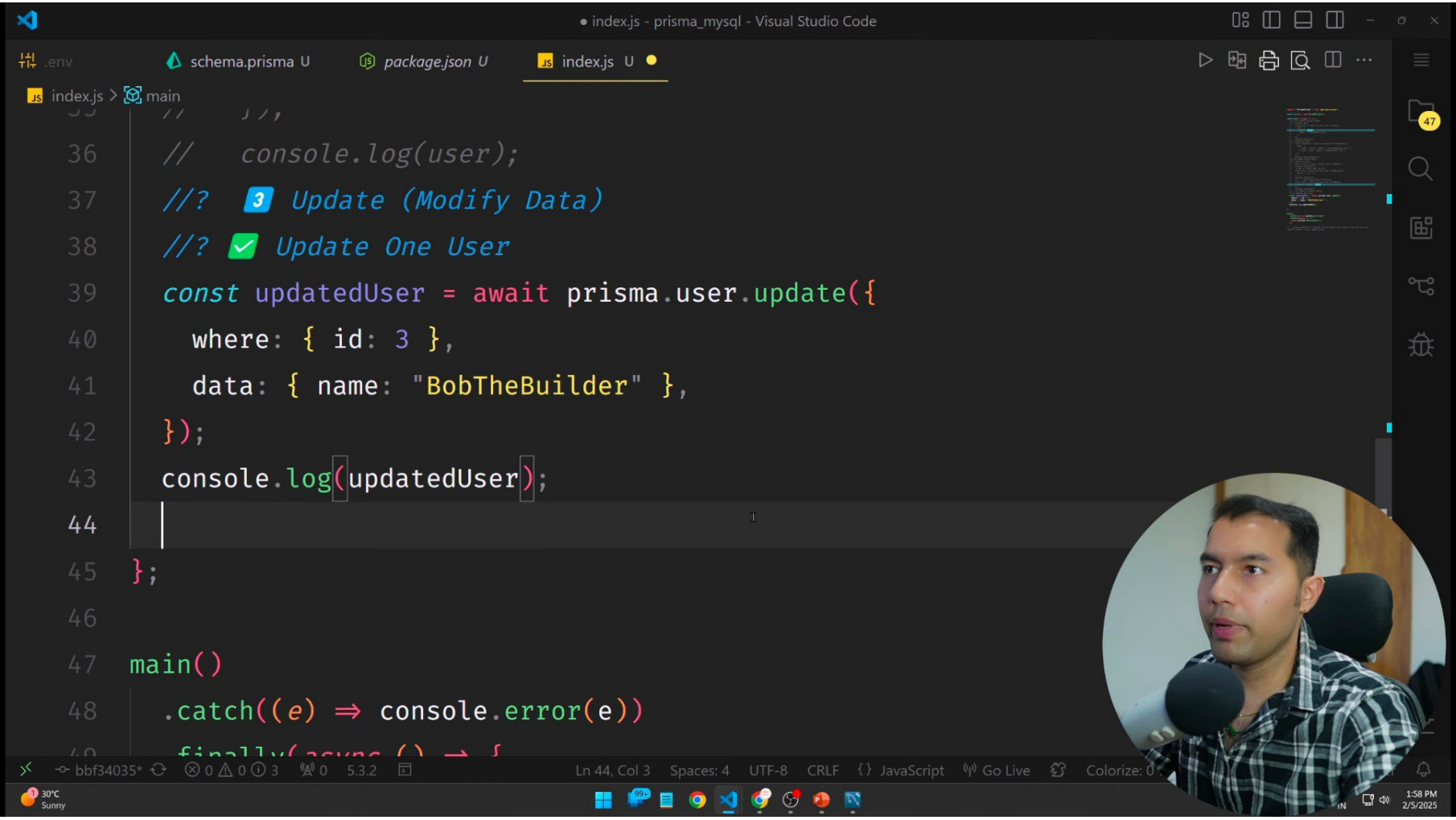
Ln 29, Col 22 | Spaces: 4 | UTF-8 | CRLF | JavaScript | Go Live

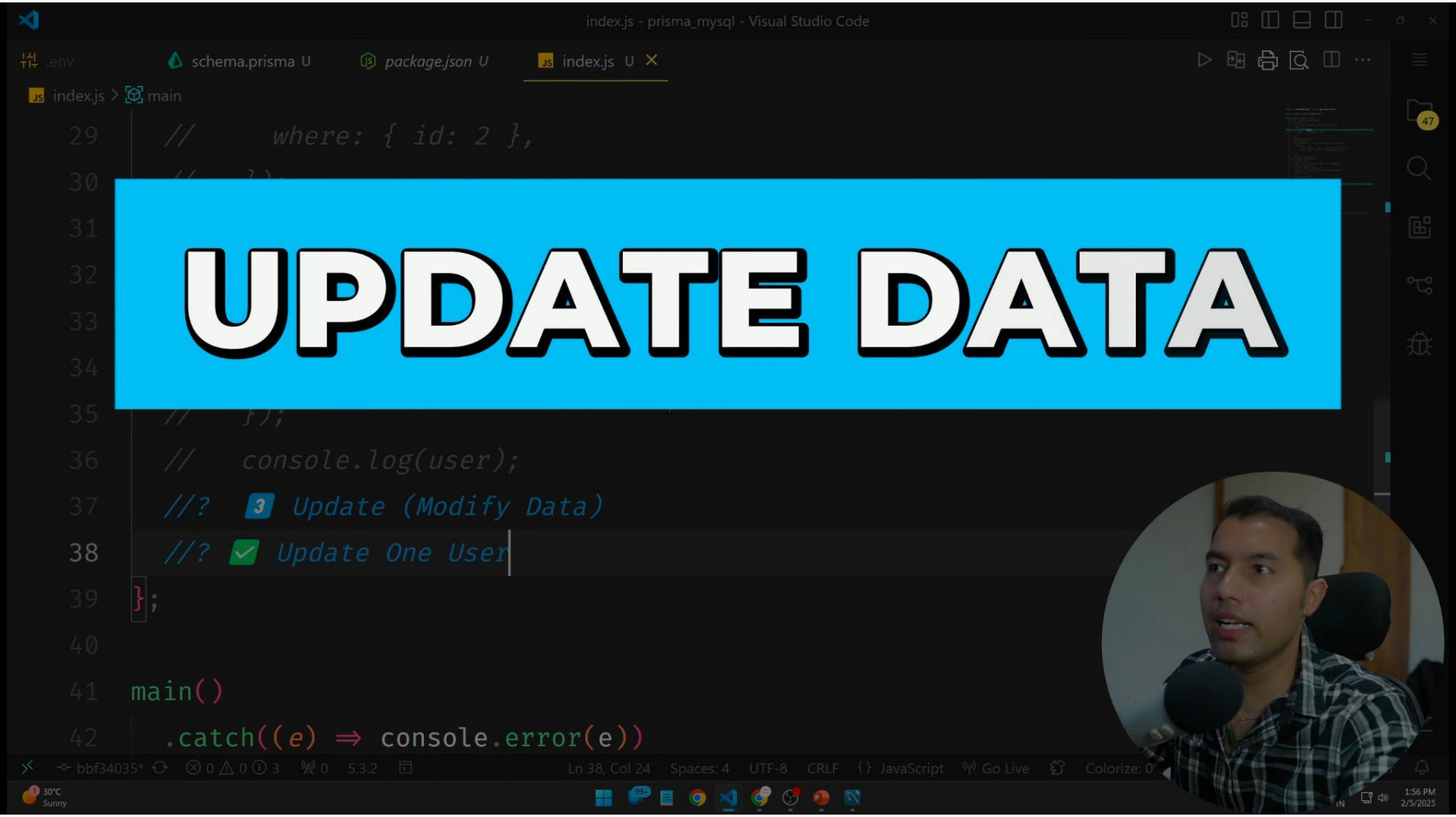
Finance headline
European shares...

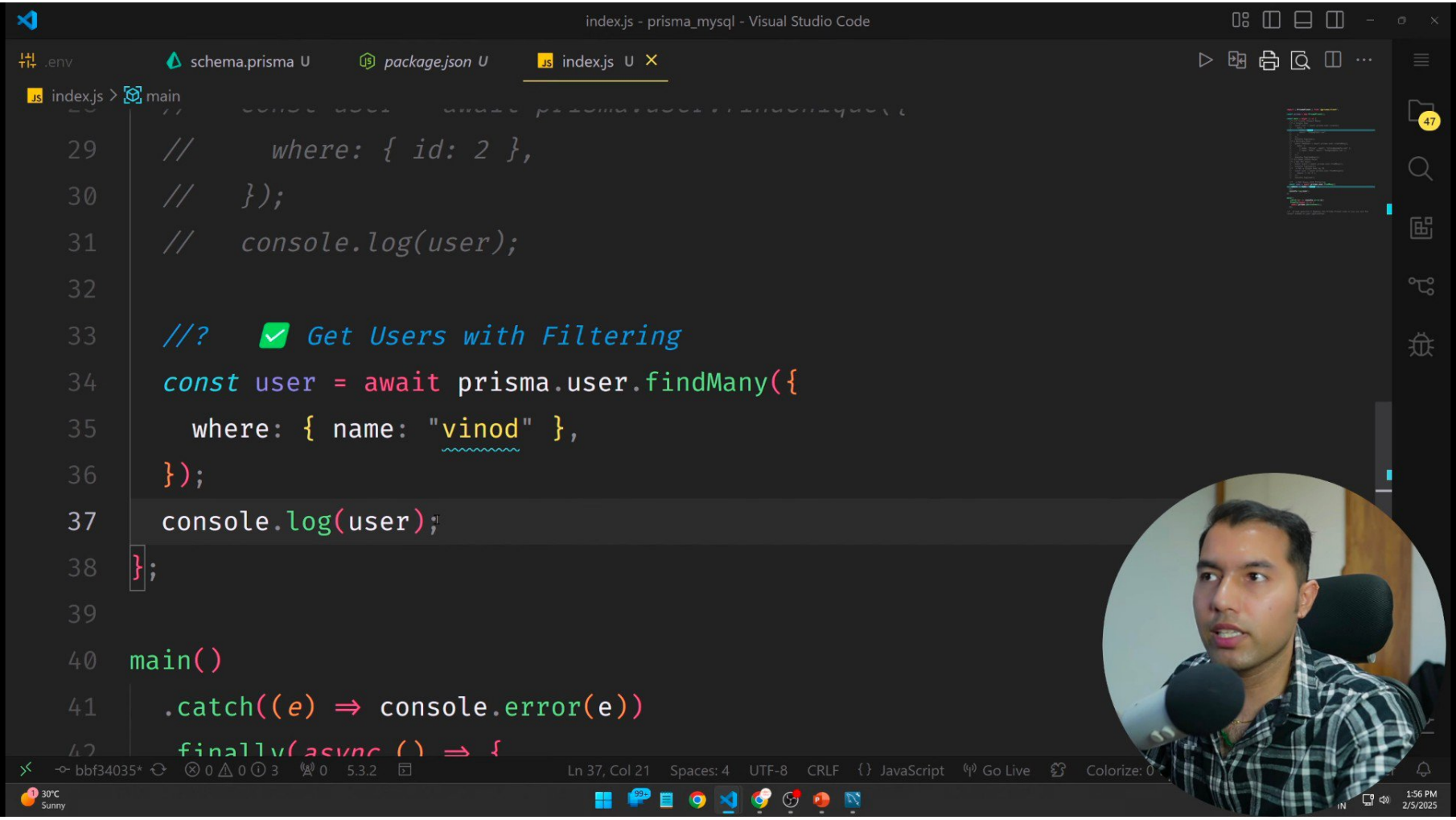
1:53 PM
2/5/2025

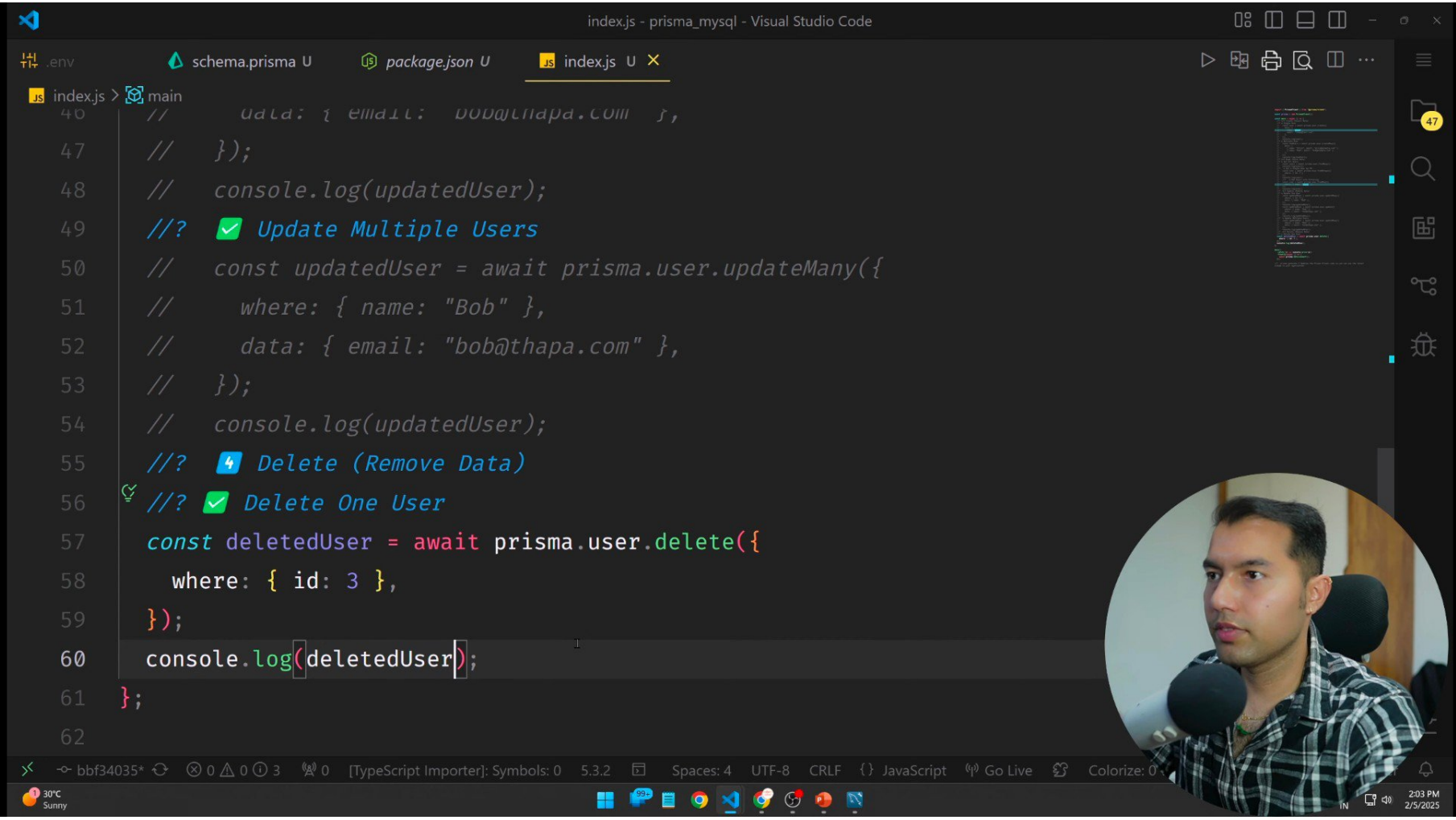

● index.js - prisma_mysql - Visual Studio Code
.env
schema.prisma U
package.json U
index.js U
index.js > main
// });
// console.log(newUsers);
//? 2 Read (Fetch Data)
//? ✅ Get All Users
const users = await prisma.user.findMany();
console.log(users);
};
main()
.catch((e) => console.error(e))
.finally(async () => {
await prisma.$disconnect();
});
bbf34035* 0 0 2 0 5.3.2
Ln 26, Col 21 Spaces: 4 UTF-8 CRLF JavaScript Go Live
Finance headline
European shares...
1:52 PM
2/5/2025


index.js - prisma_mysql - Visual Studio Code
.env
schema.prisma U
package.json U
index.js U
index.js > main
// });
// console.log(user);
// },
// });
// console.log(newUsers);
};
main()
.catch((e) => console.error(e))
.finally(async () => {
await prisma.$disconnect();
READ DB DATA
Ln 23, Col 1
Spaces: 4
UTF-8
CRLF
JavaScript
Go Live
Finance headline
European shares...
1:51 PM
2/5/2025

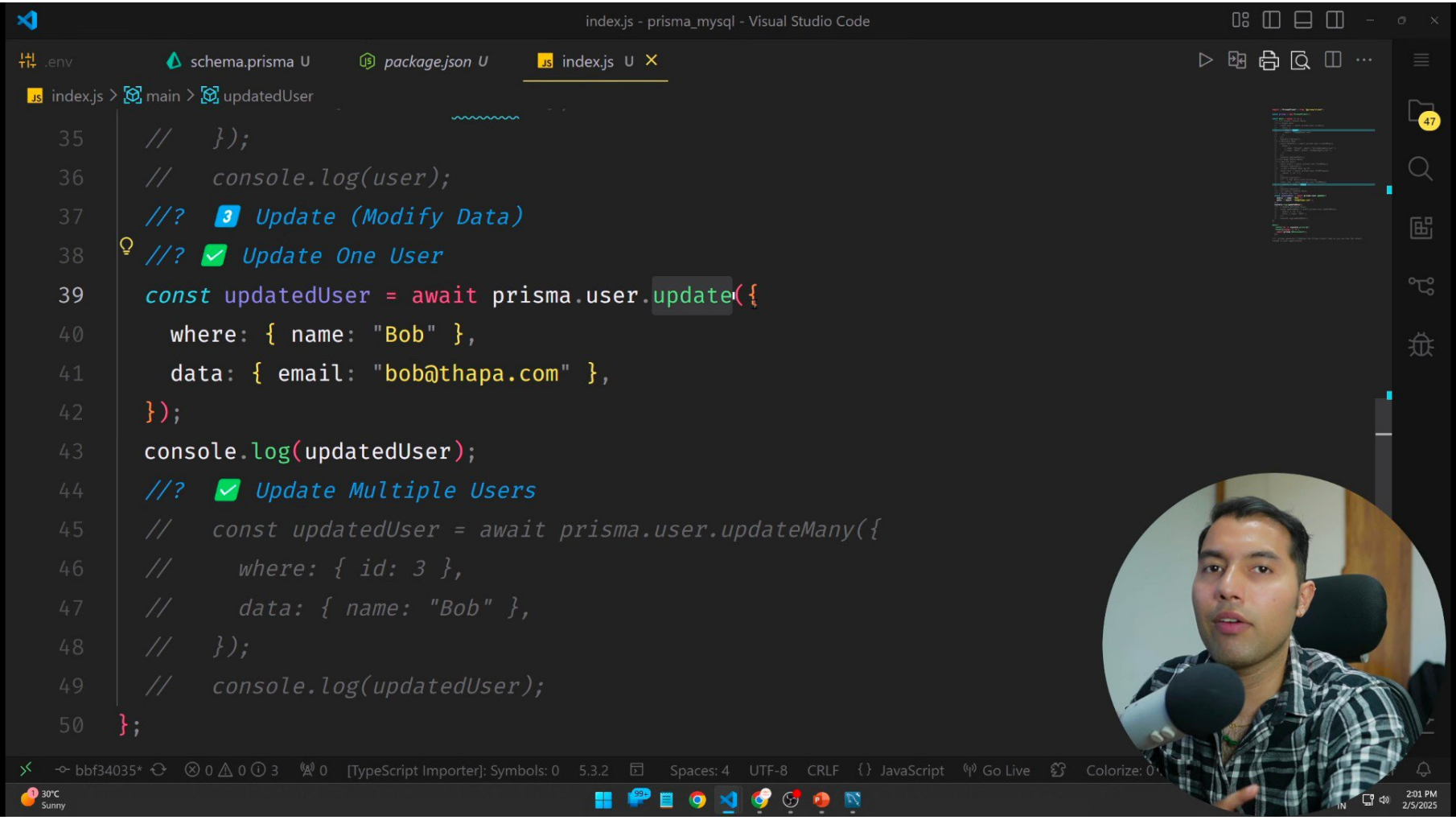

● index.js - prisma_mysql - Visual Studio Code
.env
schema.prisma U
package.json U
index.js U
index.js > main
//   console.log(user);
//? 3 Update (Modify Data)
//? ✅ Update One User
const updatedUser = await prisma.user.update({
where: { id: 3 },
data: { name: "BobTheBuilder" },
});
console.log(updatedUser);
};
main()
.catch((e) => console.error(e))
Ln 44, Col 3
Spaces: 4
UTF-8
CRLF
JavaScript
Go Live
30°C
Sunny
1:58 PM
2/5/2025


index.js - prisma_mysql - Visual Studio Code
schema.prisma U
package.json U
index.js U
index.js > main
// where: { id: 2 },
// });
// console.log(user);
//? 3 Update (Modify Data)
//? ✅ Update One User
};
main()
.catch((e) ⇒ console.error(e))
UPDATE DATA
Ln 38, Col 24
Spaces: 4
UTF-8
CRLF
JavaScript
Go Live


index.js - prisma_mysql - Visual Studio Code
.env
schema.prisma U
package.json U
index.js U
index.js > main
// where: { id: 2 },
// });
// console.log(user);
//? ✅ Get Users with Filtering
const user = await prisma.user.findMany({
where: { name: "vinod" },
});
console.log(user);
};
main()
.catch((e) => console.error(e))
Ln 37, Col 21
Spaces: 4
UTF-8
CRLF
JavaScript
Go Live
30°C
Sunny
1:56 PM
2/5/2025


index.js - prisma_mysql - Visual Studio Code
.env
schema.prisma U
package.json U
index.js U
index.js > main
//   data: { email: "bob@thapa.com" },
//   });
//   console.log(updatedUser);
//? ✅ Update Multiple Users
//   const updatedUser = await prisma.user.updateMany({
//   where: { name: "Bob" },
//   data: { email: "bob@thapa.com" },
//   });
//   console.log(updatedUser);
//? 🔷 Delete (Remove Data)
//? ✅ Delete One User
const deletedUser = await prisma.user.delete({
  where: { id: 3 },
});
console.log(deletedUser);
};
bbf34035*
[TypeScript Importer]: Symbols: 0
5.3.2
Spaces: 4
UTF-8
CRLF
JavaScript
Go Live
30°C
Sunny
2:03 PM
2/5/2025


index.js - prisma_mysql - Visual Studio Code
schema.prisma U
package.json U
index.js U
index.js > main
DELETE DATA
-10
//? Delete (Remove Data)
//? Delete One User
main()
.catch((e) => console.error(e))


index.js - prisma_mysql - Visual Studio Code
.env
schema.prisma U
package.json U
index.js U
index.js > main > updatedUser
35 // });
36 // console.log(user);
37 //? 3 Update (Modify Data)
38 //? Update One User
39 const updatedUser = await prisma.user.update({
40 where: { name: "Bob" },
41 data: { email: "bob@thapa.com" },
42 });
43 console.log(updatedUser);
44 //? Update Multiple Users
45 // const updatedUser = await prisma.user.updateMany({
46 // where: { id: 3 },
47 // data: { name: "Bob" },
48 // });
49 // console.log(updatedUser);
50 };
bbf34035*
[TypeScript Importer]: Symbols: 0
5.3.2
Spaces: 4
UTF-8
CRLF
JavaScript
Go Live
30°C
Sunny
2:01 PM
2/5/2025

4. Which databases are supported by Prisma ORM?

Answer:

Prisma supports:

- PostgreSQL
- MySQL
- SQLite
- SQL Server
- MongoDB
- CockroachDB

5. How do you install Prisma in a Node.js project?

Answer:

Run the following commands:

bash Copy code

```
npm install prisma --save-dev
npx prisma init
```

This will create a `prisma` folder with a `schema.prisma` file and a `.env` file for your database URL.

6. What is `schema.prisma`?

Answer:

`schema.prisma` is the main configuration file of Prisma.

It defines:

- The database connection (`datasource`)
- The Prisma client generator
- The data models (tables)

Example:

prisma Copy code

```
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

generator client {
  provider = "prisma-client-js"
}

model User {
  id    Int     @id @default(autoincrement())
  name  String
  email String  @unique
}
```

7. How do you generate the Prisma client?

Your ankit pawar a fresher give me all interviews questions and answers for the prisma orm

Sure, Ankit 👍
Here's a complete list of Prisma ORM interview questions and answers — designed especially for freshers like you who want to confidently explain Prisma in interviews (for Full Stack or Backend roles).

🧠 Basic Prisma ORM Interview Questions and Answers

1. What is Prisma ORM?

Answer:
Prisma is a next-generation Object Relational Mapper (ORM) for Node.js and TypeScript. It helps developers interact with databases easily by auto-generating type-safe queries instead of writing raw SQL.

Example:

```ts
const users = await prisma.user.findMany();
```

This fetches all users from the database — simple and clean!

---

2. What are the main components of Prisma?

Answer:
Prisma has three main components:

1. Prisma Client – Auto-generated and type-safe query builder to access your database.
2. Prisma Migrate – Handles schema changes and database migrations.
3. Prisma Studio – A GUI tool to view and edit your data visually.

---

3. What are the advantages of using Prisma?

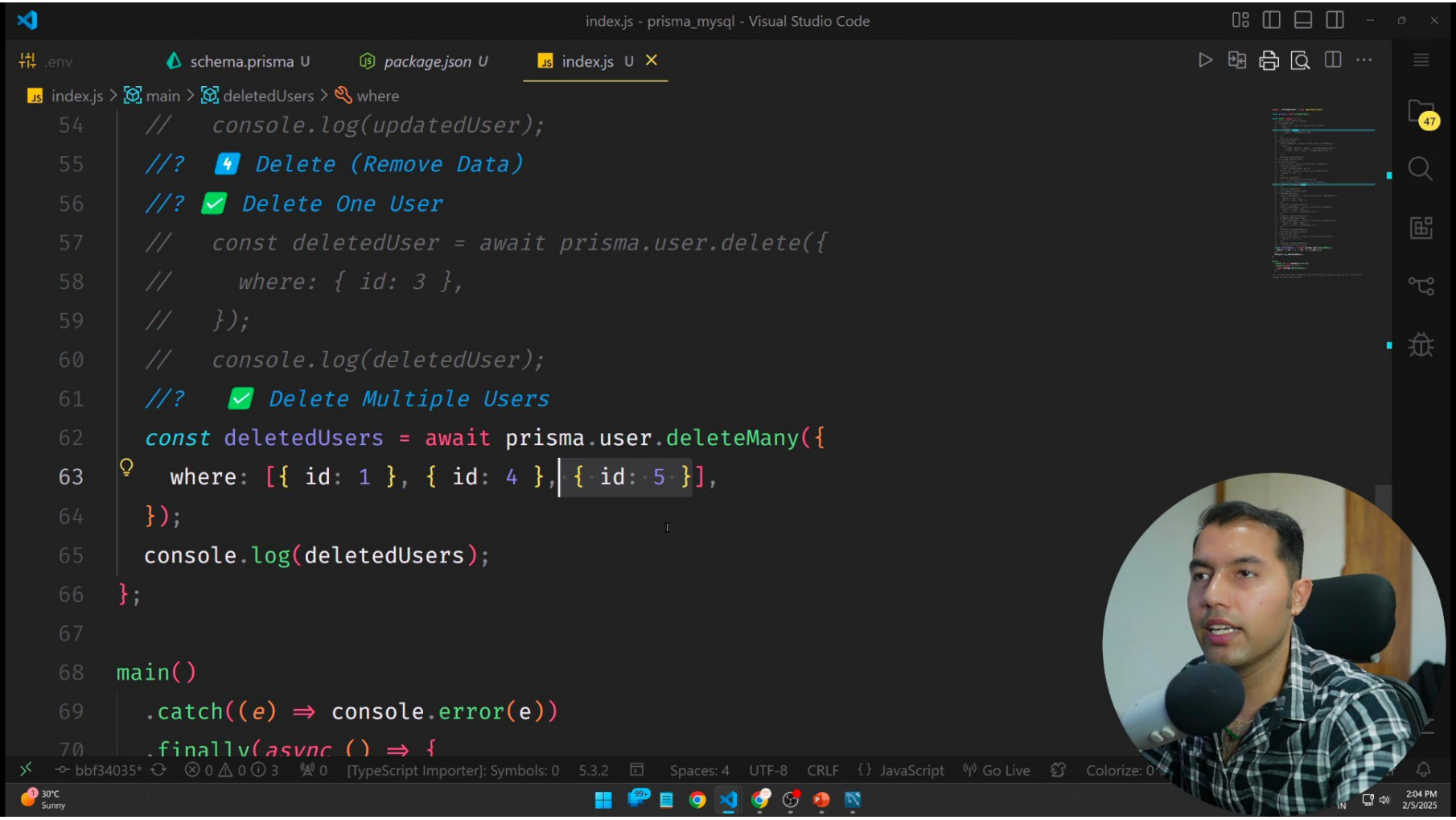
Answer:
✅ Type safety with TypeScript
✅ Easy to use and auto-completion in IDE
✅ Better productivity and clean syntax
✅ Simplified database migrations
✅ Works with multiple databases (PostgreSQL, MySQL, SQLite, MongoDB, etc.)

---

4. Which databases are supported by Prisma ORM?

Answer:
Prisma supports:

- PostgreSQL
- MySQL
- SQLite
- SQL Server


index.js - prisma_mysql - Visual Studio Code
.env
schema.prisma U
package.json U
index.js U
index.js > main > deletedUsers > where
// console.log(updatedUser);
//? Delete (Remove Data)
//? Delete One User
// const deletedUser = await prisma.user.delete({
// where: { id: 3 },
// });
// console.log(deletedUser);
//? Delete Multiple Users
const deletedUsers = await prisma.user.deleteMany({
where: [{ id: 1 }, { id: 4 }, { id: 5 }],
});
console.log(deletedUsers);
};
main()
.catch((e) => console.error(e))
.finally(async () => {
bbf34035*
[TypeScript Importer]: Symbols: 0
5.3.2
Spaces: 4
UTF-8
CRLF
JavaScript
Go Live
30°C
Sunny
2:04 PM
2/5/2025

14. How do you update and delete data using Prisma?

Answer:

✅ Update:

```ts
await prisma.user.update({
  where: { id: 1 },
  data: { name: "Ankit Pawar" },
});
```

✅ Delete:

```ts
await prisma.user.delete({
  where: { id: 1 },
});
```

---

15. How do you handle transactions in Prisma?

Answer:

```ts
await prisma.$transaction([
  prisma.user.create({ data: { name: "A" } }),
  prisma.post.create({ data: { title: "Post A" } }),
]);
```

Both queries succeed or fail together.

🔒 Advanced / Conceptual Questions

16. How does Prisma ensure type safety?

Answer:

Prisma automatically generates a TypeScript client based on your schema.

This means queries like `prisma.user.findMany()` know the exact fields and types at compile-time.

---

17. What happens when you change the Prisma schema?

Answer:

When you edit `schema.prisma`:

1. Run `npx prisma migrate dev` → updates the database
2. Run `npx prisma generate` → updates the Prisma client

---

18. What is the `@relation` attribute in Prisma?

Answer:

`@relation` defines relationships between tables (models), like foreign keys in SQL.

⚙️ Intermediate Prisma Questions

11. What is the difference between Prisma Migrate and Prisma Generate?

Answer:

- Prisma Migrate: Creates and applies changes to the database schema.
- Prisma Generate: Generates the Prisma client based on the schema.

---

12. How do you handle relations in Prisma?

Answer:
You can define one-to-many, many-to-many, and one-to-one relationships.

Example (one-to-many):

prisma Copy code

```prisma
model User {
  id     Int      @id @default(autoincrement())
  name   String
  posts  Post[]
}

model Post {
  id       Int    @id @default(autoincrement())
  title    String
  author   User?  @relation(fields: [authorId], references: [id])
  authorId Int?
}
```

---

13. How do you perform filtering or conditions in Prisma?

Answer:

ts Copy code

```ts
const users = await prisma.user.findMany({
  where: { name: { contains: "Ankit" } },
});
```

---

14. How do you update and delete data using Prisma?

Answer:
✅ Update:

ts Copy code

```ts
await prisma.user.update({
  where: { id: 1 },
  data: { name: "Ankit Pawar" },
});
```

✅ Delete:

ts Copy code

7. How do you generate the Prisma client?

Answer:
After defining your models, run:

```bash
npx prisma generate
```

This creates the Prisma Client that you can import and use in your code.

---

8. How do you run Prisma migrations?

Answer:
Run:

```bash
npx prisma migrate dev --name init
```

This applies schema changes to your database and updates the migration history.

---

9. How do you use Prisma Client in a project?

Answer:

```ts
import { PrismaClient } from '@prisma/client';
const prisma = new PrismaClient();

async function main() {
  const allUsers = await prisma.user.findMany();
  console.log(allUsers);
}

main();
```

---

10. What is Prisma Studio used for?

Answer:
Prisma Studio is a visual database explorer — it lets you view, edit, and delete data directly in your browser.
To open it:

```bash
npx prisma studio
```

⚙️ Intermediate Prisma Questions

11. What is the difference between Prisma Migrate and Prisma Generate?

Answer:

- Prisma Migrate: Creates and applies changes to the database schema.

Chetanya sharma • 3rd+
Full Stack Software Developer | Laravel ...
3mo •

+ Follow

✅ 🎯 15 Basic Prisma Interview Questions Every Backend Developer Should Know (With Answers)

📌 1. What is Prisma in Node.js?
Answer: Prisma is an open-source next-generation ORM for Node.js and TypeScript that helps developers write type-safe database queries using an auto-generated query builder.

📌 2. Which databases does Prisma support?
Answer: Prisma supports PostgreSQL, MySQL, SQLite, SQL Server, CockroachDB, and MongoDB.

📌 3. What is the schema.prisma file?
Answer: It's the central configuration file that defines your database models, datasource (DB connection), and generator (like Prisma Client).

📌 4. How do you generate Prisma Client?
Answer: By running npx prisma generate. It reads schema.prisma and generates a fully-typed Prisma Client.

📌 5. How to migrate your schema to the database?
Answer: Use npx prisma migrate dev --name init to create and apply a new migration based on your schema changes.

📌 6. What is Prisma Client?
Answer: A type-safe query builder auto-generated from your Prisma schema to interact with your database using JavaScript/TypeScript.

📌 7. How do you fetch all users using Prisma Client?
Answer: const users = await prisma.user.findMany();

Add a comment...

@

I chose Prisma because it makes database handling easier, type-safe, and faster.
It's beginner-friendly and integrates well with frameworks like Next.js and Express.js.

---

22. How is Prisma better than traditional ORMs like Sequelize?

Answer:

- Prisma provides auto-generated, type-safe queries
- Easier migrations
- Modern syntax
- Better integration with TypeScript

---

23. How can you connect Prisma with your PostgreSQL database?

Answer:
Edit `.env`:

ini Copy code

```
DATABASE_URL="postgresql://user:password@localhost:5432/mydb"
```

Then run:

bash Copy code

```
npx prisma migrate dev
```

---

24. Can Prisma be used in Next.js projects?

Answer:
Yes, Prisma integrates perfectly with Next.js for building full-stack apps — often used with APIs and server actions.

---

25. What are some limitations of Prisma?

Answer:

- Not ideal for very complex raw SQL queries
- Limited database driver support compared to older ORMs
- Slightly heavier initial setup

Would you like me to give you a short version (20 must-know Q&A) for *quick revision before interviews* or a Prisma + Next.js interview combo Q&A (many companies ask both together)?

18. What is the `@relation` attribute in Prisma?

Answer:
`@relation` defines relationships between tables (models), like foreign keys in SQL.

---

19. Can Prisma be used with MongoDB?

Answer:
Yes. Prisma supports MongoDB, but its syntax is slightly different since MongoDB is NoSQL.

---

20. How to seed initial data in Prisma?

Answer:
You can create a `prisma/seed.ts` file and run:

bash Copy code

```
npx prisma db seed
```

Example:

ts Copy code

```
import { PrismaClient } from '@prisma/client';
const prisma = new PrismaClient();

async function main() {
  await prisma.user.create({ data: { name: "Ankit", email: "ankit@example.
}

main();
```

💬 Bonus HR-Type Questions for a Fresher

21. Why did you choose Prisma ORM?

Answer:
I chose Prisma because it makes database handling easier, type-safe, and faster.
It's beginner-friendly and integrates well with frameworks like Next.js and Express.js.

---

22. How is Prisma better than traditional ORMs like Sequelize?

Answer:

- Prisma provides auto-generated, type-safe queries
- Easier migrations
- Modern syntax
- Better integration with TypeScript

---

23. How can you connect Prisma with your PostgreSQL database?

Answer:

and apply a new migration based on your schema changes.

📌 6. What is Prisma Client?
Answer: A type-safe query builder auto-generated from your Prisma schema to interact with your database using JavaScript/TypeScript.

📌 7. How do you fetch all users using Prisma Client?
Answer: const users = await prisma.user.findMany();

📌 8. How do you filter records in Prisma?
Answer: Use the where clause.

```
const activeUsers = await prisma.user.findMany({
  where: { status: 'active' }
});
```

📌 9. How do you relate models in Prisma?
Answer: By defining relationships in the schema like this:

```
model Post {
  id     Int     @id @default(autoincrement())
  title  String
  author User    @relation(fields: [authorId], references: [id])
  authorId Int
}
```

📌 10. How to create a user using Prisma?
Answer:

```
await prisma.user.create({
  data: {
    name: 'Chetan Sharma',
    email: 'chetan@example.com',
  },
});
```

📌 11. How do you update a record in Prisma?

});

📌 11. How do you update a record in Prisma?
Answer: await prisma.user.update({
  where: { id: 1 },
  data: { name: 'Updated Name' },
});

📌 12. How to delete a record using Prisma?
Answer: await prisma.user.delete({
  where: { id: 1 },
});

📌 13. What is prisma studio?
Answer: Prisma Studio is a GUI for your database. You can run it using npx prisma studio.

📌 14. How do you handle nested writes (create with relations)?
await prisma.user.create({
  data: {
    name: 'John',
    posts: {
      create: [{ title: 'First Post' }],
    },
  },
});

📌 15. What are @id, @default, and @relation in Prisma?
Answer:
@id: Marks a field as primary key.
@default: Sets a default value.
@relation: Defines relations between models.